Google

SWE Coding Questions - 31 to 90 days ago

36 questions candidates reported | by source site, each Hard, then Medium, then Easy

Collected 2026-10-20 | one optimal answer each, brute force as pseudo-code, follow-ups

5	28	3
HARD	MEDIUM	EASY

Optimal answers: 8 from the interview site, 12 from your app, 16 written here.
★ = top 75 Medium/Hard or top 10 Easy by Google frequency (8 in this PDF).
Study tiers by Google frequency on LeetCode: TYPE 16, TALK 15, READ 5 (in this PDF).
7 design-style questions are mixed in under their topics, last in each topic and marked DESIGN-STYLE.

Order: source site, recency (last 30 days, 90 days, ...), Hard, Medium, Easy, then strictly newest to oldest

- **1Point3Acres (17): Hard 0, Medium 16, Easy 1**
- **PracHub (8): Hard 3, Medium 4, Easy 1**
- **Exponent (11): Hard 2, Medium 8, Easy 1**

Date pills (colour = how recent): LAST 30 DAYS | LAST 90 DAYS | LAST 6 MONTHS

1Point3Acres: the date the question was last asked. PracHub: the day it was posted there. Exponent: ~ means an estimate from 'N months ago' (it is the last time any company was asked). HackTheRounds shows no per-question dates.

Topics (a tag on every dated question; headings only where there are no dates)

- BFS/DFS Graph and Tree Traversal and Shortest Paths (8)
- Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (9)
- Dynamic Programming and State-Space Optimization (6)
- Expression Parsing and Evaluation (4)
- Other: Design, Heaps, Intervals and Misc (9)

Sources

- hacktherounds.com/questions/google
- prachub.com Google SWE Coding & Algorithms, Technical Screen only
- aced.io/questions?company=google&role=swe&type=coding
- 1point3acres.com/interview/problems/company/google (SWE coding, no OA)

How to use

- Tap any title in the Contents to jump to it.
- Every page has < Prev | Contents | Next > at the bottom.
- Prev and Next move one question at a time; Contents returns here.

Google - Contents

36 questions | tap a title to jump to it

1POINT3ACRES

17 questions | last 30 days: 0 | last 90 days: 17 Go >

ASKED: 31 TO 90 DAYS AGO (17) Go >

Medium (16) Go >

	M	1. Temperature Monitoring Data Structure	DESIGN / OTHER		Sep 10 '26	READ | 1Point3Acres
	M	2. Minimum-Cost Tree Disconnection	BFS / DFS / TREE		Sep 9 '26	TALK | 1Point3Acres
	M	3. Rotten Oranges / Multi-Source BFS (taxis)	BFS / DFS / TREE	IN APP	Sep 8 '26	TYPE | LC 994 | 1Point3Acres
	M	4. Longest Same-Color Grid Sequence	DP		Sep 8 '26	TALK | LC 562 | 1Point3Acres
	M	5. Compute File System Total Size	BFS / DFS / TREE		Sep 2 '26	READ | 1Point3Acres
	M	6. Next-Word Predictor	DESIGN / OTHER		Sep 2 '26	READ | 1Point3Acres
	M	7. Implement Python's itertools.tee	DESIGN / OTHER		Aug 20 '26	READ | 1Point3Acres
	M	8. Cutting Ribbons (LC 1891)	ARRAY / STRING		Aug 16 '26	TALK | LC 1891 | 1Point3Acres
	M	9. Union of Sorted Interval Lists	ARRAY / STRING		Aug 16 '26	TALK | LC 986 | 1Point3Acres
	M	10. Nested List Weight Sum (LC 339) with Follow-...	EXPRESSION		Aug 16 '26	TALK | LC 339 | 1Point3Acres
	M	11. Target Expressions from Ordered Digits	EXPRESSION		Aug 13 '26	TALK | LC 282 | 1Point3Acres
	M	12. Progressive Array Partitioning	ARRAY / STRING		Aug 11 '26	TALK | 1Point3Acres
	M	13. 2D Points Connected Components by Radius	BFS / DFS / TREE		Aug 4 '26	TALK | LC 547 | 1Point3Acres
	M	14. Streaming Median (LC 295) with Follow-ups	DESIGN / OTHER	IN APP	Aug 3 '26	TYPE | LC 295 | 1Point3Acres
★	M	15. Decode String	EXPRESSION	IN APP	Jul 29 '26	TYPE | LC 394 | 1Point3Acres
	M	16. Batch and Streaming Log Deduplication	DESIGN / OTHER		Jul 23 '26	READ | 1Point3Acres

Easy (1) Go >

	E	17. Robot Status Message Deduplication Window	DESIGN / OTHER		Sep 10 '26	TYPE | LC 359 | 1Point3Acres

PRACHUB

8 questions | last 30 days: 0 | last 90 days: 8 Go >

POSTED: 31 TO 90 DAYS AGO (8) Go >

Hard (3) Go >

	H	18. Repeatedly Delete Leaves Matching a Target V...	BFS / DFS / TREE	IN APP	Sep 16 '26	TALK | LC 1325 | PracHub
	H	19. Smallest Missing Positive Integer in Linear Tim...	ARRAY / STRING	IN APP	Sep 9 '26	TYPE | LC 41 | PracHub
	H	20. Find the Longest Straight Run of One Color	DP		Aug 23 '26	TALK | LC 562 | PracHub

Medium (4) Go >

	M	21. Sliding-Window Mean After Removing the Larg...	ARRAY / STRING		Sep 1 '26	TALK | LC 480 | PracHub
	M	22. Implement a Reusable Locker Allocation System	DESIGN / OTHER		Aug 14 '26	TALK | LC 1845 | PracHub
	M	23. Count Prefix-Matching Simple Paths in a Label...	BFS / DFS / TREE		Aug 9 '26	TYPE | LC 79 | PracHub
★	M	24. Expand Recursive Key-Value Placeholders in a...	EXPRESSION		Aug 4 '26	TYPE | LC 394 | PracHub

Easy (1) Go >

	E	25. Check an Interval Against Existing Intervals for...	ARRAY / STRING	IN APP	Aug 8 '26	TALK | LC 729 | PracHub

EXPONENT

11 questions | last 30 days: 0 | last 90 days: 11 Go >

ASKED (ROUGH): 31 TO 90 DAYS AGO (11) Go >

Hard (2) Go >

★	H	26. Implement LRU Cache.	DESIGN / OTHER	IN APP	~Sep '26	TYPE | LC 146 | Exponent

Google - Contents

36 questions | tap a title to jump to it

	H	27. Merge k sorted linked lists.	DESIGN / OTHER	IN APP	~Aug '26	TYPE | LC 23 | Exponent
Medium (8)						**Go >**
★	M	28. Course Schedule	BFS / DFS / TREE	IN APP	~Sep '26	TYPE | LC 207 | Exponent
	M	29. Construct a binary tree given its preorder and in...	BFS / DFS / TREE	IN APP	~Sep '26	TALK | LC 105 | Exponent
★	M	30. Find the maximum subarray sum.	DP	IN APP	~Sep '26	TYPE | LC 53 | Exponent
	M	31. Partition an array into two sub-arrays with equal...	DP	IN APP	~Sep '26	TYPE | LC 416 | Exponent
	M	32. Permutations	DP	IN APP	~Sep '26	TYPE | LC 46 | Exponent
★	M	33. Merge Intervals	ARRAY / STRING	IN APP	~Aug '26	TYPE | LC 56 | Exponent
★	M	34. Group anagrams	ARRAY / STRING	IN APP	~Aug '26	TYPE | LC 49 | Exponent
★	M	35. Given n houses in a line with money in each, fin...	DP	IN APP	~Aug '26	TYPE | LC 198 | Exponent
Easy (1)						**Go >**
	E	36. Given an array of strings, return the words that...	ARRAY / STRING		~Sep '26	TALK | LC 500 | Exponent

SIDE NOTE: marked problems that are not in this book

★ The top 75 Medium and Hard problems and the top 10 Easy problems by Google frequency are marked in this book. These 39 are marked too but are not questions here. Do them from your app. Rank is the position in the Medium and Hard list, or E1 to E10 for Easy.

Rank	LC	Problem	Lvl	Freq
3	3	Longest Substring Without Repeating	M	73/69.3
5	42	Trapping Rain Water	H	73.6/55.1
7	128	Longest Consecutive Sequence	M	70.3/58.3
8	15	3Sum	M	72.6/58.3
9	7	Reverse Integer	M	66.3/51.4
15	11	Container With Most Water	M	68.3/41.1
18	410	Split Array Largest Sum	H	57.7/46.9
21	17	Letter Combinations of a Phone Number	M	57.8/51.4
25	239	Sliding Window Maximum	H	54.9/46.9
26	31	Next Permutation	M	61/41.1
30	6	Zigzag Conversion	M	53.5/51.4
31	34	Find First and Last Position	M	53/46.9
32	54	Spiral Matrix	M	59.3/33
34	1944	Number of Visible People in a Queue	H	53.6/41.1
37	33	Search in Rotated Sorted Array	M	57.2/33
38	84	Largest Rectangle in Histogram	H	56.8/33
39	167	Two Sum II	M	49.8/46.9
42	155	Min Stack	M	50.8/41.1
43	19	Remove Nth Node From End of List	M	50.2/41.1
44	424	Longest Repeating Character Replacement	M	53.6/33
47	739	Daily Temperatures	M	47.8/41.1
48	78	Subsets	M	51.7/33
49	39	Combination Sum	M	47.6/41.1
52	16	3Sum Closest	M	50.6/33
53	131	Palindrome Partitioning	M	46/41.1
54	209	Minimum Size Subarray Sum	M	45.2/41.1
57	402	Remove K Digits	M	44.1/0
62	151	Reverse Words in a String	M	46/33
63	312	Burst Balloons	H	41.8/0
67	51	N-Queens	H	58.6/0
75	210	Course Schedule II	M	36.5/33
E1	20	Valid Parentheses	E	67.3/70.9
E3	14	Longest Common Prefix	E	73.6/55.1
E4	88	Merge Sorted Array	E	69.7/55.1
E5	26	Remove Duplicates from Sorted Array	E	65.7/55.1
E7	28	Find the Index of the First Occurrence	E	56.2/61
E8	485	Max Consecutive Ones	E	56/58.3
E9	169	Majority Element	E	66.5/33
E10	242	Valid Anagram	E	57.3/51.4

Freq is LeetCode's frequency for Google: all time / last 30 days.

1POINT3ACRES

17 questions | Hard 0, Medium 16, Easy 1 | asked in the last 30 days: 0, last 90 days: 17 (grouped by recency, newest first) | topics: BFS/DFS and trees first

ASKED: 31 TO 90 DAYS AGO (17)

Medium (16)

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 1 of 36

Temperature Monitoring Data Structure

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 223/228**

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

QUESTION

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days. SWEdata-structurestreamingsliding-windowFrequencySingle reportLast asked2026-09-10Stageonsite-coding

APPROACH

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the window is the last N readings or the last N days, what moving average should return before the window fills, and the number of readings.

BRUTE FORCE TO SAY OUT LOUD

Store everything in a list and compute the window average and max by scanning. O(window) per query.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

For a count window keep a deque of the last N readings and a running sum for the average, and a monotonic decreasing deque for the maximum: pop smaller values from the back on insert and drop the front when it expires. All operations O(1) amortized. For time windows store (time, value) and evict by timestamp.

DRY-RUN THESE CASES

N = 1, duplicate maxima, negative readings, the maximum expiring, fewer readings than the window.

LIKELY FOLLOW-UPS

Minimum too, median over a window, or out-of-order timestamps.

PITFALLS

Removing the wrong element from the max deque when equal values repeat. Floating-point drift in the running sum.

HOW TO THINK ABOUT IT

Three operations on the last N readings. The average only needs a running sum, and the maximum needs a monotonic deque that drops smaller values from the back as larger ones arrive, so the front is always the max of the window. Decide first whether the window is a count of readings or a span of time.

Basis: Written; a sliding window with an average (LeetCode 346) and a window maximum (LeetCode 239), here as one data structure.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
readings = []
def insert(x):
    readings.append(x)
def average():
    window = readings[-N:]
    return sum(window) / len(window)
def maximum():
    return max(readings[-N:])
```

TIME	SPACE
O(N) per average or max	O(N)

WHY IT IS SLOW

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

OPTIMAL SOLUTION · Running sum plus a monotonic deque

TIME	SPACE
O(1) amortized per operation	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

FOLLOW-UP 2 Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

FOLLOW-UP 3 Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Temperature Monitoring Data Structure
#
# Implement temperature insertion, a moving average, and a
# constant-time maximum query for a recent window. Clarify
# whether the window is based on reading count or elapsed
# days.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The window is the last D days (elapsed time), not the
# last N readings.
# 2. Also support minimum, or the median.
# 3. Floating-point drift in the running sum.
#
# INTERVIEW NOTES
# Three operations on the last N readings. The average
# only needs a running sum, and the maximum needs a
# monotonic deque that drops smaller values from the back
# as larger ones arrive, so the front is always the max of
# the window. Decide first whether the window is a count
# of readings or a span of time.
#
# BRUTE FORCE (say it first): Keep a list and scan the
# window on every query
# Store every reading and compute sum and max over the
# last N each time. Correct and the right oracle; the cost
# is a full window scan for each query. Time O(N) per
# average or max. Space O(N).


from collections import deque


# BRUTE FORCE (say it first): Keep a list and scan the
# window on every query | time O(N) per average or max
class MonitorBrute:
    def __init__(self, n):
        self.n, self.data = n, []

    def insert(self, x):
        self.data.append(x)
```

```
    def average(self):
        w = self.data[-self.n :]
        return sum(w) / len(w)
```

```
    def maximum(self):
        return max(self.data[-self.n :])


# HELPERS for the base question
class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# FOLLOW-UP 2: Also support minimum, or the median.
# Answer: Minimum uses a second monotonic deque with the
# opposite comparison. A median needs two heaps with lazy
# deletion or a sorted container, O(log N) per update,
# because it is not decomposable like sum and max.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
```

```
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
        self.maxq.drop_while(too_old)
        self.minq.drop_while(too_old)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()


# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
```

```python
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
# is full.
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)


# FOLLOW-UP 1: The window is the last D days (elapsed
# time), not the last N readings.
# Answer: Store (timestamp, value) pairs and evict from
```

```python
# the front while the timestamp is at or before now minus
# the span, in both the window and the max deque.
# Everything else is unchanged.
class TimedMonitor(SlidingStats):
    """Window = readings from the last `span` days."""

    def __init__(self, span):
        super().__init__()
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        # drop readings at or before day - span
        cutoff = day - self.span
        self.expire(lambda key: key <= cutoff)


if __name__ == "__main__":
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    # window is [1, 4, 2]
    assert m.maximum() == 4 and m.minimum() == 1
    assert m.average() == 7 / 3
    m.insert(0)
    # window is [4, 2, 0]
    assert m.maximum() == 4 and m.minimum() == 0
    t = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        t.insert(day, x)
    # day 1 is two days old, so it left the window
    assert t.maximum() == 5 and t.average() == 4
```

NOTES FROM CANDIDATES

The introductory description refers to the most recent N days, while the API requirements refer to the most recent N readings. Clarify the window definition before coding; the day-based variant does not specify how timestamps enter the interface. Empty-window behavior and insertion complexity are unspecified.

For the reading-count window with positive N, retain the last N readings in a FIFO queue and maintain their sum. Add the new reading and subtract any expired reading. Agree whether a partially filled window returns its current mean or is unavailable. Maintain a second deque of reading indices with nonincreasing temperatures. Expire old indices at the front and remove smaller or equal temperatures from the back before adding the new index. The front holds the current maximum; indices distinguish duplicate readings.

Average and maximum queries are O(1). Each reading enters and leaves each deque at most once, so insertion is amortized O(1) but can take O(N) for one update; auxiliary space is O(N). This implements the reading-count contract. A day-based contract additionally needs timestamps, expiration timing, and an ordering rule.

PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

MEDIUM | 1Point3Acres | phone-screen | 2 of 36

Minimum-Cost Tree Disconnection

ASKED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 197/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

QUESTION

EXAMPLES

Negative-weight follow-up

```
A --(-1)--> B --(-1)--> C
Minimum total cut cost: -2
```

Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

APPROACH

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what must be disconnected (root from every leaf), whether the tree is rooted and directed, whether weights can be negative, and what to return for an empty or single-node tree.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of edges and keep those that separate the root from all leaves. Exponential.

OPTIMAL IDEA

Define best(v) as the cheapest set of cuts in v's subtree that disconnects v from every leaf below it. For each child c with edge weight w, either cut the edge, paying w, or keep it and require best(c) (not allowed if c is a leaf). Sum over children. With non-negative weights that is the answer. With negative weights, once you cut an edge the whole subtree below becomes free, and you may still cut any negative edge there, so the cut option costs w plus the sum of min(0, edge) over that subtree.

DRY-RUN THESE CASES

A single edge, the negative chain A to B to C (answer -2), a leaf at different depths, a star.

LIKELY FOLLOW-UPS

Negative weights, many roots, or recursion depth for a chain tree.

PITFALLS

Treating every negative edge as a mandatory cut. Recursing deeply on a chain.

HOW TO THINK ABOUT IT

On a tree, every decision about a subtree depends only on the edge above it, so define a function of the node that returns the best cost inside its subtree. For each child you either cut the edge above it (pay its weight) or keep it and make the child itself disconnected from the leaves below. With negative weights, a cut edge frees the whole subtree, so you may cut every negative edge in it for free gain.

Basis: Written. Assumed setup (confirm it): a rooted tree with edge weights (cuts cost the weight, weights may be negative); cut edges so the root is disconnected from every leaf; a leaf is a non-root node with no children; with no leaves the cost is 0.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, edges):
    best = INF
    for k in range(len(edges) + 1):
        for cut in combinations(edges, k):
            tree = remove_edges(edges, cut)
            if not reaches_leaf(tree, root):
                cost = sum(w for _, _, w in cut)
                best = min(best, cost)
    return best
```

TIME	SPACE
O(2^E * n)	O(n)

WHY IT IS SLOW

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

OPTIMAL SOLUTION · Tree DP: best(v) = cheapest cuts inside v's subtree that disconnect v from its leaves

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

FOLLOW-UP 2 The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

FOLLOW-UP 3 Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimum-Cost Tree Disconnection
#
# Examples: Negative-weight follow-up: A --(-1)--> B
# --(-1)--> C Minimum total cut cost: -2 Cutting both
# edges costs -2, even though cutting just the first edge
# already separates the root from the original leaf.
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Weights are non-negative.
# 2. The tree is a long chain and recursion overflows.
# 3. Return which edges to cut.
#
# INTERVIEW NOTES
# On a tree, every decision about a subtree depends only
# on the edge above it, so define a function of the node
# that returns the best cost inside its subtree. For each
# child you either cut the edge above it (pay its weight)
# or keep it and make the child itself disconnected from
# the leaves below. With negative weights, a cut edge
# frees the whole subtree, so you may cut every negative
# edge in it for free gain.
#
# BRUTE FORCE (say it first): Try every subset of edges
# Enumerate each subset of edges to cut, check that the
# root can reach no leaf, and keep the cheapest.
# Exponential, but exact, so it checks the DP. Time O(2^E
# * n). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first): Try every subset of edges |
# time O(2^E * n)
def disconnect_brute(n, edges):
    children = defaultdict(list)
    for u, v, w in edges:
        children[u].append(v)
    leaves = [
        v
        for v in range(n)
        if v != 0 and not children[v]
    ]
    best = None
    for mask in range(1 << len(edges)):
```

```
        kept = [
            (u, v)
            for i, (u, v, w) in enumerate(edges)
            if not mask >> i & 1
        ]
        adj = defaultdict(list)
        for u, v in kept:
            adj[u].append(v)
        seen, stack = {0}, [0]
        while stack:
            x = stack.pop()
            for y in adj[x]:
                if y not in seen:
                    seen.add(y)
                    stack.append(y)
        if not any(leaf in seen for leaf in leaves):
            cost = sum(
                w
                for i, (u, v, w) in enumerate(edges)
                if mask >> i & 1
            )
            best = (
                cost
                if best is None
                else min(best, cost)
            )
    return best if best is not None else 0


# HELPERS for the base question
INF = float("inf")
```

```
def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    # FOLLOW-UP 2: a loop, not recursion: safe on a long
    # chain
    # breadth-first order: parents come before children
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
```

```
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free


# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free(c)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(c), which is impossible when c is
# a leaf.
# FOLLOW-UP 1: Weights are non-negative.
# Answer: Then free(c) is always 0 and the recurrence
# collapses to best(v) = sum over children of (w if the
# child is a leaf, else min(w, best(child))).
# FOLLOW-UP 2: The tree is a long chain and recursion
# overflows.
# Answer: Compute nodes in reverse BFS order (children
# before parents) and store best and free in arrays, no
# recursion needed. State the space as O(n).
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]


# FOLLOW-UP 3: Return which edges to cut.
# Answer: Remember at each child which option was cheaper,
# then walk down from the root collecting the cut edges.
# Cut edges also include every negative edge below any cut
# edge.
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts, stack = [], [0]
```

```
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            keep = best[c] if kids[c] else INF
            if w + free[c] <= keep:
                # cut here, plus negative edges below c
                cuts.append((v, c))
                below = [c]
                while below:
                    x = below.pop()
                    for y, wy in kids[x]:
                        if wy < 0:
                            cuts.append((x, y))
                        below.append(y)
```

```
            else:
                stack.append(c)
    return cuts


if __name__ == "__main__":
    # the question's chain A -(-1)-> B -(-1)-> C
    neg = [(0, 1, -1), (1, 2, -1)]
    assert disconnect_cost(3, neg) == -2
    cuts = sorted(edges_to_cut(3, neg))
    assert cuts == [(0, 1), (1, 2)]
    # non-negative costs: cut the cheaper edge
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3
    assert edges_to_cut(3, chain) == [(1, 2)]
    # a single node needs no cut
    assert disconnect_cost(1, []) == 0
    # a long chain does not overflow the stack
    big = [(i, i + 1, 1) for i in range(5000)]
    assert disconnect_cost(5001, big) == 1
```

NOTES FROM CANDIDATES

Clarify the required result for an empty tree and a single-node tree. No final return convention is specified for these cases. Include recursion-stack memory in space analysis. The tree can degenerate into a chain; balanced-tree depth is not a worst-case bound.

Coding took place directly in Google Docs with no supplied framework; the candidate also had to define the tree node structure.

The interviewer asked the candidate to think aloud while developing the approach.

For nonnegative weights, use postorder DP. Let F(v) be the minimum cost to disconnect node v from every original leaf below it. An original leaf has F(v) = infinity; an internal node has F(v) = sum(min(weight(v,c), F(c))) over its children. Each branch must be blocked independently. Use a safe infinity representation rather than adding to a fixed-width maximum integer.

For negative weights, cutting an ancestor edge does not remove the opportunity to cut negative edges below it. Track A(v), the minimum optional cut cost when the subtree is already disconnected: A(v) = sum(min(0, weight(v,c)) + A(c)), with A(leaf) = 0. Then use F(v) = sum(min(weight(v,c) + A(c), F(c))), retaining F(leaf) = infinity. This includes beneficial redundant negative cuts

without double counting edges.

Both recurrences process each edge once: O(n) time and O(h) recursive auxiliary space, where h can equal n. A single original root-leaf cannot be disconnected from itself by cutting edges; agree on its result convention before coding.

PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 3 of 36

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 0** | **PRIORITY 67/228**

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app: LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

EXAMPLES

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges
    variant): return number of minutes until all
    reachable 1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

APPROACH

Drill multi-source BFS from memory in under 10 min.

Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.

Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

Basis: LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Rotten Oranges / Multi-Source BFS (taxis)
#
# Examples: Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-
# oranges variant): return number of minutes until all
# reachable 1s become 2 (4). Taxi variant: for each empty
# cell, distance to nearest taxi.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Sources move or new taxis appear. How do you keep the
# distances fresh?
# 2. Why is multi-source BFS the same cost as single-
# source BFS?
# 3. Signal variant: a signal spreads from sources, how
# long until every target gets it?
# 4. A signal spreads from sources to targets: how long
# until every target is reached?
#
# INTERVIEW NOTES
# 'Spread outward from several starting points at the same
# time and count the rounds' is multi-source BFS: put
# every source in the queue before you start, and process
# the queue one round at a time. Rounds are minutes.
# Always count the fresh ones first so that unreachable
# oranges are detected at the end.
#
# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute
# Each minute, find every fresh orange next to a rotten
# one and rot them all at once, until nothing changes.
# Easy to reason about and the oracle; each minute touches
# the entire grid. Time O(minutes * m * n). Space O(m *
# n).


from collections import deque


# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute | time O(minutes * m * n)
def oranges_by_scanning(grid):
    g = [row[:] for row in grid]
    minutes = 0
    while True:
        rot = [
            (r, c)
```

```
            for r in range(len(g))
            for c in range(len(g[0]))
            if g[r][c] == 1
            and any(
                0 <= r + dr < len(g)
                and 0 <= c + dc < len(g[0])
                and g[r + dr][c + dc] == 2
                for dr, dc in (
                    (1, 0),
                    (-1, 0),
                    (0, 1),
                    (0, -1),
                )
            )
        ]
        if not rot:
            break
        for r, c in rot:
            g[r][c] = 2
        minutes += 1
    return -1 if any(1 in row for row in g) else minutes


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
```

```
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
            for r in range(rows)]
```

```
# FOLLOW-UP 4: A signal spreads from sources to targets:
# how long until every target is reached?
# Answer: One multi-source BFS from the sources, then the
# answer is the largest distance over the targets (-1 if
# any target is unreachable, 0 if there are no targets).
# Per-source BFS would cost O(mn * sources).
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)
```

```
if __name__ == "__main__":
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0
```

NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).
The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.
For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

MEDIUM | 1Point3Acres | phone-screen | 4 of 36

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 148/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

Basis: LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Same-Color Grid Sequence
#
# Given a grid of colors, return the longest consecutive
# same-color sequence along a horizontal, vertical, or
# diagonal line. Input is a two-dimensional grid of colors
# such as R, G, and B. Return the length of the longest
# consecutive sequence of a single color horizontally,
# vertically, or diagonally. Examples: R R R G R R B B G R
# R R B G B R Answer: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. The line may also bend (a connected region).
#
# INTERVIEW NOTES
# A straight line can run in four directions (right, down,
# down-right, down-left). For each cell and each direction
# keep the length of the same-colour run that ends there:
# it is the predecessor's run plus one if the colours
# match, otherwise 1. Because every run only looks one
# cell back in a fixed direction, one pass in reading
# order is enough.
#
# BRUTE FORCE (say it first): Extend a line from every
```

```
# cell in every direction
# Start at each cell and walk in each of the four
# directions while the colour matches. Correct; it re-
# walks the same cells many times. Time O(m * n * max(m,
# n)). Space O(1).


# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction | time O(m * n * max(m, n))
def longest_line_brute(g):
    best = 0
    for i in range(len(g)):
        for j in range(len(g[0])):
            for di, dj in (
                (0, 1),
                (1, 0),
                (1, 1),
                (1, -1),
            ):
                k, a, b = 0, i, j
                while (
                    0 <= a < len(g)
```

```
                    and 0 <= b < len(g[0])
                    and g[a][b] == g[i][j]
                ):
                    k += 1
                    a += di
                    b += dj
                best = max(best, k)
    return best


# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
```

```
# region).
# Answer: Then it is a different problem: the largest
# connected same-colour region, found with BFS/DFS or
# union-find in O(m * n). The run-length DP only handles
# straight lines because each run depends on exactly one
# predecessor.
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.

Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 5 of 36

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 204/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

BRUTE FORCE TO SAY OUT LOUD

Recursive DFS adding file sizes. Simple and usually the expected first answer.

OPTIMAL IDEA

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

DRY-RUN THESE CASES

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

LIKELY FOLLOW-UPS

Cache totals, handle updates, or find the largest directory.

PITFALLS

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute File System Total Size
#
# Examples: root = Dir( Dir(File(10), File(20)), File(5),
# Dir(File(30)), ) Total: 65
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.
#
# BRUTE FORCE (say it first): Plain recursion
# Sum a file's size, or the sum over a directory's
# children. Correct for ordinary trees and the answer most
# interviews want first. It overflows on very deep trees
# and loops forever on a cycle. Time O(n). Space O(depth)
# call stack.


import heapq


# HELPERS for the brute force
class File:
    def __init__(self, size):
        self.size, self.parent = size, None


# BRUTE FORCE (say it first): Plain recursion | time O(n)
def total_size_recursive(node):
    if isinstance(node, File):
        return node.size
    return sum(
        total_size_recursive(c) for c in node.children
    )
```

```
# HELPERS for follow-up 3
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
            node.cached = sum(size_of(c) for c in kids)
    return root.cached


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
```

```
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total


# FOLLOW-UP 1: The tree is queried often and files change.
# Answer: Cache each directory's total and, when a file
# changes, clear the cache only for its ancestors (walk
# parent links). A query recomputes only cleared nodes.
# Updates cost O(depth).
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def directory_totals(root):
    # the size of every directory, in one pass
    cached_total(root)
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
```

```
            stack.extend(node.children)
    return out
```

```python
# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))


if __name__ == "__main__":
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 6 of 36

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 224/228**

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

BRUTE FORCE TO SAY OUT LOUD

Store all sentences and scan them for each prediction. O(corpus) per query.

OPTIMAL IDEA

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

DRY-RUN THESE CASES

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

LIKELY FOLLOW-UPS

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

PITFALLS

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	**O(corpus size)**

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	**O(number of distinct pairs)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Next-Word Predictor
#
# Process training data and predict the most frequent next
# word for a supplied word. Maintain prediction state
# during processing so prediction can run in constant
# time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top K predictions.
# 2. Use two words of context (trigrams).
# 3. The corpus keeps changing and old text should fade.
#
# INTERVIEW NOTES
# Prediction in O(1) means the answer must already be
# sitting in a table when the query arrives, so do the
# work while training. Keep counts per (word, next word)
# and, for each word, the current best successor. Each
# count only increases by one, so the best can be updated
# by comparing just the one count that changed.
#
# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction
# Keep the sentences and count successors of the query
# word from scratch on each call. Simple and correct, but
# every query is as slow as one full pass over the data.
# Time O(corpus size) per prediction. Space O(corpus
# size).


import heapq
from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction | time O(corpus size) per prediction
class PredictorBrute:
    def __init__(self):
        self.sentences = []

    def train(self, sentence):
        self.sentences.append(sentence.split())

    def predict(self, word):
        c = Counter(
            s[i + 1]
```

```
            for s in self.sentences
            for i in range(len(s) - 1)
            if s[i] == word
        )
```

```
        return (
            min(c, key=lambda w: (-c[w], w))
            if c
            else None
        )


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
# FOLLOW-UP 1: Return the top K predictions.
# Answer: Keep the Counter per word and use
# heapq.nsmallest on (-count, word): O(d log K) per query,
# or keep a sorted structure per word if queries dominate.
# FOLLOW-UP 2: Use two words of context (trigrams).
# Answer: The key becomes the pair (previous two words)
# and everything else is identical, at the price of more
# memory. Back off to the bigram table when the pair was
# never seen.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)
```

```
    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None

    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
```

```
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
```

```
    q.train("a b")
    assert q.predict("a") == "b"
```

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.

The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.

After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.

With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.

Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.

Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 7 of 36

Implement Python's itertools.tee

ASKED 2026-08-20 | 61 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 225/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement the behavior of Python's **`tee`**: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

APPROACH

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the source iterator can be consumed by one consumer first, what happens if one consumer is far ahead, and whether thread safety is needed.

BRUTE FORCE TO SAY OUT LOUD

Store the whole source in a list and give each consumer an index. Simple, but defeats the purpose for large or infinite iterators.

OPTIMAL IDEA

Keep a deque per consumer. When a consumer needs an item and its deque is empty, pull the next item from the source and append it to every consumer's deque. Each item is held only until all consumers have read it. Build with a shared state object and a small iterator class with __iter__ and __next__. The linked-list variant uses one chain of nodes with each consumer holding a pointer, which frees nodes as the slowest consumer moves.

DRY-RUN THESE CASES

Two consumers at different speeds, n = 1, n = 0, an empty source, an exhausted source.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Thread safety (a lock around the pull), and memory growth if one consumer stalls.

PITFALLS

Pulling from the source once per consumer instead of once per item. Not appending to the other deques.

HOW TO THINK ABOUT IT

The source can only be consumed once, so someone must remember items until every consumer has read them. The state to keep is exactly 'items the slowest consumer has not read yet', so either give each consumer a queue that the leader fills, or keep one shared linked list with a pointer per consumer so memory is freed automatically.

Basis: The itertools.tee documentation's recipe; the linked-list variant is how CPython shares buffers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Python's itertools.tee
#
# Implement the behavior of Python's tee: accept one
# iterator and an integer n, then return n independent
# iterators over the same source sequence. Define the
# required function and classes yourself and improve a
# naive design toward an efficient shared-state
# implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Memory grows when one consumer stalls. Is there a way
# to free items automatically?
# 2. Make it safe for several threads.
# 3. Why not just copy the iterator?
#
# INTERVIEW NOTES
# The source can only be consumed once, so someone must
# remember items until every consumer has read them. The
# state to keep is exactly 'items the slowest consumer has
# not read yet', so either give each consumer a queue that
# the leader fills, or keep one shared linked list with a
# pointer per consumer so memory is freed automatically.
#
# BRUTE FORCE (say it first): Materialize the whole source
# into a list
# Read everything into a list and give each consumer an
# index. Correct for finite data, but it consumes the
# source eagerly, never works on an infinite iterator and
# holds all items in memory. Time O(n) up front. Space
# O(n).


import threading
from collections import deque


# FIRST IDEA (simplest): copy into a list
def tee_naive(iterable, n=2):
    # simplest idea: copy everything into a list
    items = list(iterable)
    # but it never works for endless iterators
    return tuple(iter(items) for _ in range(n))


# BASE QUESTION (optimal): One deque per consumer, filled
```

```
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)


# FOLLOW-UP 1: Memory grows when one consumer stalls. Is
# there a way to free items automatically?
# Answer: Use one shared singly linked list with a pointer
# per consumer. The only references to old nodes are the
# consumers' pointers, so Python's garbage collector frees
# nodes behind the slowest consumer without any
# bookkeeping.
class Cell:
    # one link of the shared list
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# FOLLOW-UP 2: Make it safe for several threads.
# Answer: Pull from the source under one lock and append
# to the queues inside it. Without the lock two consumers
# can pull two items and append them in different orders
# to different queues.
def tee(iterable, n=2):
```

```
    source = iter(iterable)
    # one lock keeps the pulls in a single order
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    # the first reader pulls the value
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value, cell = cell.value, cell.next
            yield value

    # every reader starts at the same first cell
    head = Cell()
    return tuple(consumer(head) for _ in range(n))


if __name__ == "__main__":
    a, b = tee(range(5))
    # readers are independent, at any speed
    assert next(a) == 0 and next(a) == 1
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    x, y, z = tee_queues("abc", 3)
```

```
assert list(x) == list(y) == list(z) == list("abc")
# works on an endless source, which tee_naive cannot
def count():
    i = 0
    while True:
        yield i
        i += 1
e1, e2 = tee(count())
assert [next(e1) for _ in range(3)] == [0, 1, 2]
assert next(e2) == 0
assert [list(i) for i in tee_naive([1, 2], 2)] == [
    [1, 2], [1, 2]]
```

NOTES FROM CANDIDATES

The round depends on Python iterator-protocol fluency as much as general data-structure reasoning. Clarify laziness, source exhaustion, invalid n, and whether concurrent consumption is in scope before implementing.
A correct baseline consumes the source lazily once and gives each returned iterator its own queue: when one queue is empty, pull one source value, append it to every queue, then return it from the requesting queue. If n iterators trail across a gap of g values, however, those queues can retain O(n·g) buffered copies. For the optimal shared-state design, keep one linked buffer (or indexed deque) of source values plus one cursor per iterator, and discard a buffered value only after every cursor has passed it. That preserves independent consumption with O(g+n) auxiliary state, where g is the distance between the fastest and slowest consumers.

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

MEDIUM | 1Point3Acres | onsite-coding | 8 of 36

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 19.6** **30 DAYS 0** **PRIORITY 147/228**

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEarraybinary-searchcomplexity-analysismedium FrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

BRUTE FORCE TO SAY OUT LOUD

Try every length from the maximum down to 1 and count pieces. O(n * max).

OPTIMAL IDEA

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

DRY-RUN THESE CASES

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

LIKELY FOLLOW-UPS

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

PITFALLS

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Cutting Ribbons (LC 1891)
#
# LC 1891 Cutting Ribbons: given wood lengths and K,
# maximize the equal piece length that yields at least K
# pieces. Follow-ups cover the search bound, floating-
# point lengths, and why complexity depends on the value
# range rather than the number of elements.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the running time depend on the value range
# and not just on the number of ribbons?
# 2. What if the candidate length is given to you?
# 3. What is a good upper bound for the search?
# 4. Give the search a tighter upper bound.
#
# INTERVIEW NOTES
# 'Largest length such that at least K pieces fit' is a
# maximisation with a yes/no check. Larger lengths yield
# fewer pieces, so the check is monotone and you can
# binary search the answer. The check itself is one pass:
# sum of ribbon // length.
#
# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down
# For each length L from the maximum ribbon down to 1
# count the pieces and return the first L that gives at
# least K. Correct, but with ribbons up to 10^9 it would
# try a billion lengths. Time O(n * max). Space O(1).


# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down | time O(n * max)
def max_length_brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        if sum(r // L for r in ribbons) >= k:
            return L
    return 0


# HELPERS for the base question
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)
```

```
# FOLLOW-UP 4: Give the search a tighter upper bound.
# Answer: No piece can be longer than the longest ribbon,
# and K pieces of length L need K * L total length, so L
# <= sum // K. Starting hi at min(max(ribbons), sum // K)
# saves a few probes and also returns 0 immediately when
# sum < K.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
```

```
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best


# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
    return lo
```

```
if __name__ == "__main__":
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied? Expect the follow-ups, not the base implementation, to consume most of the round.
For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

MEDIUM | 1Point3Acres | onsite-coding | 9 of 36

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 139/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

BRUTE FORCE TO SAY OUT LOUD

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

OPTIMAL IDEA

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

DRY-RUN THESE CASES

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

LIKELY FOLLOW-UPS

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

INTERVIEW-DAY PLAN

PITFALLS

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Union of Sorted Interval Lists
#
# Reverse of LC 986: given two individually sorted,
# internally non-overlapping interval lists, return their
# union. Follow-ups extend to K lists with a heap, inputs
# too large for memory, and lists that may overlap
# internally.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the lists are too big for memory?
# 2. What if one list may overlap itself?
# 3. Do touching intervals merge?
# 4. Each list may overlap itself or be unsorted.
#
# INTERVIEW NOTES
# Each list is already sorted, so you can walk both with
# two pointers like the merge step of merge sort: always
# take the interval with the smaller start next. Once the
# intervals come out in start order, merging is the usual
# one-pass fold: extend the last output interval if the
# next one overlaps or touches it, otherwise start a new
# one.
#
# BRUTE FORCE (say it first): Concatenate, sort and merge
# Put all intervals in one list, sort by start, and fold
# overlapping ones. Correct and short; the sort ignores
# that both inputs were already sorted. Time O((m + n) log
# (m + n)). Space O(m + n).


import heapq


# BRUTE FORCE (say it first): Concatenate, sort and merge
# | time O((m + n) log (m + n))
def union_sort(a, b):
    out = []
    for s, e in sorted(a + b):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

```
# HELPERS for the base question
# FOLLOW-UP 1: What if the lists are too big for memory?
# Answer: Read each list through an iterator (a file or a
# paged API). union_k only ever holds one interval per
```

```python
# list plus the current merged interval, so memory is O(K)
# and it can emit results as it goes.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))
```

```python
# FOLLOW-UP 2: What if one list may overlap itself?
# Answer: The two-pointer merge already works, because it
# folds every incoming interval into the last output
# interval.
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# FOLLOW-UP 4: Each list may overlap itself or be
# unsorted.
# Answer: Normalise each list first (sort and merge it by
# itself, O(n log n)), after which the two-pointer union
# applies. Or sort everything together if memory allows.
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))


if __name__ == "__main__":
    a = [(1, 3), (6, 8)]
    b = [(2, 4), (9, 10)]
    assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
    # closed intervals that touch merge
    assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
    # half-open intervals that only touch stay apart
    assert union_two([(1, 3)], [(3, 5)], False) == [
        (1, 3), (3, 5)]
    # three lists, one of them empty
    three = union_k([a, b, []])
    assert three == [(1, 4), (6, 8), (9, 10)]
    # a list that overlaps itself, and an unsorted one
    merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
    assert merged == [(1, 12)]
    assert union_unsorted([(9, 10), (1, 2)], []) == [
        (1, 2), (9, 10)]
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap. For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

MEDIUM | 1Point3Acres | onsite-coding | 10 of 36

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 156/228

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2
    + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and
    the final-number flush.
```

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

BRUTE FORCE TO SAY OUT LOUD

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

OPTIMAL IDEA

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

DRY-RUN THESE CASES

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

LIKELY FOLLOW-UPS

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

PITFALLS

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×
2
# + 14×2 + 2×2 + 91×3 = 320. Use [2, [91]] as a manual
# trace for bracket depth and the final-number flush.
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).
#
# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth
# json.loads turns the text into nested lists; a recursive
# function adds each integer times its depth. Clear, but
# it builds the whole structure and recurses as deep as
# the nesting. Time O(n). Space O(depth).


import json


# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth | time O(n)
def depth_sum_parsed(s):
    def go(items, depth):
        return sum(
            (
                x * depth
                if isinstance(x, int)
                else go(x, depth + 1)
            )
            for x in items
        )
    return go(json.loads(s), 1)


# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total


# FOLLOW-UP 2: What is the inverse version (deeper
# integers weigh less)?
# Answer: That is LeetCode 364. Do a level-by-level pass
# that keeps a running sum of everything seen so far and
# add it to the total once per level: total += running_sum
# on each level.
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# less, LeetCode 364).
# Answer: Process level by level and keep a running sum of
# every integer seen so far; add the running sum to the
# total once per level that holds an integer or lies above
# one (trim trailing levels that contain only empty
# lists).
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)


if __name__ == "__main__":
    data = [3, 8, [2, 14], [2, [91]]]
    # 3 + 8 + 2*2 + 14*2 + 2*2 + 91*3 = 320
    assert weight_sum(data) == 320
    text = "[3, 8, [2, 14], [2, [91]]]"
    assert weight_sum_string(text) == 320
    assert weight_sum_string("[2, [91]]") == 2 + 91 * 2
    # inverse weights: [[1,1],2,[1,1]] -> 8
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    # empty lists add nothing
```

```
    assert weight_sum([[], [[]]]) == 0
    # 20000 levels deep is still fine
    deep = []
    for _ in range(20000):
        deep = [deep, 1]
    assert weight_sum(deep) > 0
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

MEDIUM | 1Point3Acres | onsite-coding | 11 of 36

Target Expressions from Ordered Digits

ASKED 2026-08-13 | 68 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 103/228

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

EXAMPLES

```
For digits [2, 3, 4] and target 14, valid outputs
    include:
2 * (3 + 4)
2 + 3 * 4
```

APPROACH

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether digits may be concatenated into multi-digit numbers, which operators are allowed, whether parentheses are free, whether the digit order is fixed, and whether to return all expressions or a boolean.

BRUTE FORCE TO SAY OUT LOUD

Enumerate every operator placement and every parenthesization for the three or four digits and evaluate each. That is fine for a handful of digits and is what the first stage asks for.

OPTIMAL IDEA

Interval DP: for each interval of digits compute a map from value to the expressions that achieve it by splitting at every point and combining left and right with plus, minus and times. Memoize by interval. If only reachability is needed store just the set of values per interval, which is much smaller than the strings.

DRY-RUN THESE CASES

[2, 3, 4] with target 14 gives 2*(3+4) and 2+3*4. A target that is unreachable. A single digit. Zeros.

LIKELY FOLLOW-UPS

N digits (the DP above), division with fractions, or concatenation.

PITFALLS

Missing parentheses in the output string. Evaluating with floats.

HOW TO THINK ABOUT IT

Any expression over a run of digits is two smaller expressions joined by an operator at some split point, so the set of values for an interval is built from the sets of its two halves. Compute values bottom-up per interval and only build text for the target. Be explicit about parentheses: they come from the split structure, so every split is a candidate.

Basis: Related to LeetCode 282, Expression Add Operators, but with parentheses allowed and digits not concatenated.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

TIME	SPACE
exponential (Catalan(n) *... exponential (Catalan(n) * 3^(n-1) expressions)	**O(n)** recursion plus the value set

WHY IT IS SLOW

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

TIME	SPACE
exponential in the number... exponential in the number of results; polynomial in the number of distinct values	**O(distinct values per interval)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

FOLLOW-UP 3 Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Target Expressions from Ordered Digits
#
# Examples: For digits [2, 3, 4] and target 14, valid
# outputs include: 2 * (3 + 4) 2 + 3 * 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you handle N digits where listing every
# expression is too much?
# 2. What if concatenating digits (23) or division is
# allowed?
# 3. Only a yes/no answer is needed for many digits.
#
# INTERVIEW NOTES
# Any expression over a run of digits is two smaller
# expressions joined by an operator at some split point,
# so the set of values for an interval is built from the
# sets of its two halves. Compute values bottom-up per
# interval and only build text for the target. Be explicit
# about parentheses: they come from the split structure,
# so every split is a candidate.
#
# BRUTE FORCE (say it first): Recursively try every split
# and operator, without remembering sub-results
# Split the digits at every point, combine all value pairs
# of the two parts with +, - and *, and see whether the
# target appears. Correct, but the same sub-interval is
# recomputed many times; memoising by interval is the fix.
# Time exponential (Catalan(n) * 3^(n-1) expressions).
# Space O(n) recursion plus the value set.


from collections import defaultdict


# BRUTE FORCE (say it first): Recursively try every split
# and operator, without remembering sub-results | time
# exponential (Catalan(n) * 3^(n-1) expressions)
def _all_values(digits):
    if len(digits) == 1:
        return {digits[0]}
    out = set()
    for i in range(1, len(digits)):
        for a in _all_values(digits[:i]):
            for b in _all_values(digits[i:]):
                out |= {a + b, a - b, a * b}
    return out


def target_reachable_brute(digits, target):
    return target in _all_values(digits)


# HELPERS for the base question
def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
```

```
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))
```

```
# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
# Idea: Every expression over digits[l:r] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it.
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})


# FOLLOW-UP 3: Only a yes/no answer is needed for many
# digits.
# Answer: Store just the set of reachable values per
# interval (as target_reachable does) instead of the
# expression strings; the strings blow up long before the
# value sets do.
def target_reachable(digits, target):
    # yes/no only: keep just the set of values per range
    values = {}

    def solve(l, r):
        if (l, r) in values:
            return values[(l, r)]
        out = {digits[l]} if r - l == 1 else set()
        for m in range(l + 1, r):
            for a in solve(l, m):
                for b in solve(m, r):
                    out |= {a + b, a - b, a * b}
        values[(l, r)] = out
        return out

    return target in solve(0, len(digits))


if __name__ == "__main__":
    ans = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in ans and "2+3*4" in ans
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    # subtraction needs brackets on the right
    sub = expressions_for_target([5, 3, 1], 3)
    assert "5-(3-1)" in sub
    assert target_reachable([1] * 8, 8)
```

NOTES FROM CANDIDATES

The base case is small enough for direct enumeration, but a complicated ad hoc implementation becomes difficult to extend. Keep expression construction separate from numeric evaluation while negotiating the N-digit contract.
If the agreed contract uses every digit exactly once in order, memoize each interval [l, r) as a map from attainable numeric values to the expression strings producing them. For every split point, combine left and right entries with + and *, wrapping composite children as needed; the full-interval target bucket is the answer. Because every distinct expression must be retained, both time and storage are output-sensitive and can grow exponentially even though interval states are memoized.

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

MEDIUM | 1Point3Acres | phone-screen | 12 of 36

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 198/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	O(d)

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# Examples: Stage 1 [1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true
# because the four value-groups have different sizes. [1,
# 2, 3] -> false because every value-group has size one.
# Stage 2 [1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true. [4, 5, 6
,
# 7, 7, 8, 9] -> false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
```

```python
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
# example, which is exactly the situation where you ask.
#
# BRUTE FORCE (say it first): Compare every pair of counts
# Count each value, then check every pair of counts for
# equality. Fine for small d and a clear statement of the
# rule. Time O(d^2) where d is the number of distinct
# values. Space O(d).


from collections import Counter


# BRUTE FORCE (say it first): Compare every pair of counts
# | time O(d^2) where d is the number of distinct values
def distinct_counts_brute(nums):
    counts = list(Counter(nums).values())
    return all(
        counts[i] != counts[j]
        for i in range(len(counts))
        for j in range(i + 1, len(counts))
    )


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
```

```python
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# Answer: Do not guess. Write down what the second example
# pair implies, and check any candidate rule against BOTH.
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# HELPERS for the checks below
def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

MEDIUM | 1Point3Acres | phone-screen | 13 of 36

2D Points Connected Components by Radius

ASKED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 40.7 | 30 DAYS 0 | PRIORITY 91/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

APPROACH

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

HOW TO THINK ABOUT IT

Two points are neighbours when their squared distance is at most r squared (no square roots needed), and connectivity is transitive, so this is counting connected components: union-find or a DFS over the neighbour relation. Checking every pair is O(n^2); bucketing points into cells of side r means a point only needs to be compared with the 3 x 3 cells around it.

Basis: LeetCode 547, Number of Provinces, where two points are linked when their distance is at most r.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

OPTIMAL SOLUTION · Union-find with a spatial grid

TIME	SPACE
O(n) about when points are spread out (O(n^2) if they all share a cell)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: 2D Points Connected Components by Radius
#
# Given 2D points and radius r, connect two points when
# their distance is at most r; connectivity is transitive,
# and the output is the final number of clusters. The
# interviewer accepted or preferred a brute-force neighbor
# scan after discussing optimization ideas. Input: a list
# of 2D points and a radius r. Two points are connected if
# their Euclidean distance is <= r. Connectivity is
# transitive: if A connects to B and B connects to C, all
# three belong to the same cluster. Return the number of
# final clusters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why compare squared distances?
#
```

```python
# INTERVIEW NOTES
# Two points are neighbours when their squared distance is
# at most r squared (no square roots needed), and
# connectivity is transitive, so this is counting
# connected components: union-find or a DFS over the
# neighbour relation. Checking every pair is O(n^2);
# bucketing points into cells of side r means a point only
# needs to be compared with the 3 x 3 cells around it.
#
# BRUTE FORCE (say it first): Compare every pair and run a
# DFS
# Compare every pair of points, build the neighbour lists
# and count components with a DFS. This is the baseline
# the interviewer accepted, and it is fine for a few
# thousand points. Time O(n^2). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first): Compare every pair and run a
# DFS | time O(n^2)
def components_brute(points, r):
    n, seen, count = len(points), set(), 0
    near = (
        lambda a, b: (a[0] - b[0]) ** 2
        + (a[1] - b[1]) ** 2
        <= r * r
    )
    for s in range(n):
        if s in seen:
            continue
        count += 1
        stack = [s]
        seen.add(s)
        while stack:
            u = stack.pop()
            for v in range(n):
                if v not in seen and near(
                    points[u], points[v]
                ):
                    seen.add(v)
                    stack.append(v)
    return count


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# FOLLOW-UP 1: Why compare squared distances?
# Answer: sqrt is slow and inexact; dist <= r is the same
# as dist^2 <= r^2 for non-negative numbers, and with
# integer coordinates the comparison is exact. It also
# avoids floating-point surprises right at the boundary.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# BASELINE: compare every pair
def count_clusters_pairs(points, r):
    # the accepted baseline: compare every pair, O(n^2)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (x // r, y // r). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared.
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


if __name__ == "__main__":
    # three points joined only through the middle one
    line = [(0, 0), (3, 0), (6, 0)]
    assert count_clusters(line, 3) == 1
    assert count_clusters_pairs(line, 3) == 1
    # the far point stays alone
    far = [(0, 0), (3, 0), (7, 0)]
    assert count_clusters(far, 3) == 2
    # radius 0 only joins identical points
    same = [(1, 1), (1, 1), (2, 2)]
    assert count_clusters(same, 0) == 2
    assert count_clusters([], 5) == 0
    # both methods agree on a scatter of points
    pts = [(i * 7 % 13, i * 5 % 11) for i in range(30)]
    for r in (0, 1, 2, 3, 5):
        assert count_clusters(pts, r) == (
            count_clusters_pairs(pts, r))
```

NOTES FROM CANDIDATES

DFS, BFS, or Union-Find all fit the core connected-components model.

For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.

Clarify squared-distance comparison to avoid unnecessary square roots.

PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 14 of 36

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 0 | PRIORITY 69/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

BRUTE FORCE TO SAY OUT LOUD

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

OPTIMAL IDEA

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

DRY-RUN THESE CASES

One element, two elements, duplicates, all equal, alternating very large and very small values.

LIKELY FOLLOW-UPS

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

PITFALLS

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n)	O(n)
per insert (list shift), O(1) median	

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n)	O(n)
add, O(1) median	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Median (LC 295) with Follow-ups
#
# A near-canonical LC 295 streaming-median implementation
# using two heaps, followed by two design extensions:
# exploit a very small value domain, then support an
# arbitrary k-th-smallest query instead of only the
# median.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Almost every value is in a small range (say 0 to
# 100). Can you do better than two heaps?
# 2. Now support an arbitrary k-th smallest, not only the
# median.
# 3. What if the data stream is too large to store?
# 4. The median of only the last K values.
#
# INTERVIEW NOTES
# Keep the data split at the median: a max-heap for the
# smaller half and a min-heap for the larger half, with
# sizes equal or differing by one. Then the median is read
# from the tops. Insert by pushing through one heap into
# the other to keep every smaller value on the left, and
# rebalance the sizes.
#
# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect
# Insert each value into a sorted list with bisect.insort
# and read the middle. Very simple and the oracle; every
# insert shifts up to n elements, though in practice that
# is a fast memmove. Time O(n) per insert (list shift),
# O(1) median. Space O(n).


import bisect
import heapq
from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect | time O(n) per insert (list shift),
# O(1) median
class MedianSorted:
    def __init__(self):
        self.a = []

    def addNum(self, x):
        bisect.insort(self.a, x)

    def findMedian(self):
        n = len(self.a)
        return (
            float(self.a[n // 2])
            if n % 2
            else (self.a[n // 2 - 1] + self.a[n // 2])
            / 2
        )


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Almost every value is in a small range (say
# 0 to 100). Can you do better than two heaps?
# Answer: Yes: a counting array gives O(1) inserts, and
# the median needs one pass over at most V counts, so O(V)
# per query with V fixed. Keep a moving pointer to the
# median bucket to make queries amortized O(1).
class CountingMedian:
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1
```

```
    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value

    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2


# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# only the median.
# Answer: Two heaps only work for one fixed rank. Use an
# order-statistics structure: a Fenwick tree over
# (compressed) values, or a balanced BST augmented with
# subtree sizes.
class Fenwick:
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
```

```
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos


# FOLLOW-UP 4: The median of only the last K values.
# Answer: Keep the window in a deque (to know what
# expires) and a sorted list (to read the middle); insert
# and remove with bisect. O(K) per update because of list
# shifting; two heaps with lazy deletion give O(log K).
class WindowMedian:
    """Follow-up: median of only the last k values."""

    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2


if __name__ == "__main__":
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.add(x)
```

```
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
    assert c.median() == 3.5
```

```
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value.
For an even-sized stream, average the two heap roots.
Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.
For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.
The base problem required complete working code plus a dry run and complexity analysis.
The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 15 of 36

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 45/228

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

BRUTE FORCE

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decode String
#
# Examples: "3[a]2[bc]" -> "aaabcbc". "2[abc]3[cd]ef" ->
# "abcabccdcdcdef". "3[a2[c]]" -> "accaccacc".
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you do it recursively, and what is the
# risk?
# 2. What if the decoded string is enormous?
# 3. The decoded string would be enormous.
# 4. Return the character at position k only.
# 5. Invalid input (missing bracket, count with no
# bracket).
#
# INTERVIEW NOTES
# Nested repeats like 3[a2[c]] are a bracket structure,
# which means a stack: when you see '[' save what you have
# built and the repeat count; when you see ']' repeat the
# inner string and glue it to the saved prefix. Read
# counts as full numbers (more than one digit).
#
# BRUTE FORCE (say it first): Recursive descent over the
# string
# A function that reads letters until it meets a digit
# (then reads the number, the bracketed substring
# recursively, and repeats it) or a closing bracket. It
# mirrors the grammar and is the oracle; the recursion
# depth equals the nesting depth. Time O(output length).
# Space O(depth).


# BRUTE FORCE (say it first): Recursive descent over the
# string | time O(output length)
def decode_recursive(s):
    pos = 0

    def parse():
        nonlocal pos
        out = []
        while pos < len(s) and s[pos] != "]":
            if s[pos].isdigit():
                num = 0
                while s[pos].isdigit():
                    num = num * 10 + int(s[pos])
                    pos += 1
                pos += 1  # skip '['
                inner = parse()
                pos += 1  # skip ']'
                out.append(inner * num)
            else:
                out.append(s[pos])
                pos += 1
        return "".join(out)

    return parse()


# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
    return cur


# FOLLOW-UP 3: The decoded string would be enormous.
# Answer: Compute its length (or a given character) from
# the stack without building it: keep (length so far,
# repeat count) frames and combine as prev + inner *
# times. The length can be computed for strings that could
# never fit in memory.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total


# FOLLOW-UP 4: Return the character at position k only.
# Answer: Walk the structure with lengths: at each repeat
# reduce k modulo the inner length to find where in the
# inner string it falls (LeetCode 880 style).
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None
```

```
if __name__ == "__main__":
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")
    assert length(items) == 9
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass
```

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 16 of 36

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 89 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 226/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

APPROACH

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which entry to keep in each policy (first seen or latest timestamp), how ties on timestamp break, whether output is sorted by timestamp, and how the streaming interface returns results (get_next).

BRUTE FORCE TO SAY OUT LOUD

Sort all logs and dedupe with a set. Correct for the batch version.

OPTIMAL IDEA

Batch: keep a dict message to chosen entry (keep first-seen, or replace if the new timestamp is later) and sort the values by (timestamp, message). Streaming: a dict for membership plus a min-heap of (timestamp, message); on update of a message push a new entry and mark the old one stale (lazy deletion), and get_next pops until it finds a non-stale one. Amortized O(log n).

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The same message twice at different times, equal timestamps, an empty stream, get_next on an empty queue.

LIKELY FOLLOW-UPS

Watermarks for late data, bounded memory with eviction by age.

PITFALLS

Returning a stale heap entry. Forgetting deterministic tie-breaking.

HOW TO THINK ABOUT IT

Dedup is a dictionary keyed by the message. Sorting is only for the output order. For a stream, a heap gives the smallest timestamp quickly, but when an entry is replaced you cannot delete from the middle of a heap, so leave the stale entry and skip it when it surfaces (lazy deletion).

Basis: Written. Assumed rules (confirm them): dedupe on the message; policy 'first' keeps the earliest in input order, policy 'latest' keeps the largest timestamp; output sorted by (timestamp, message); get_next returns the retained entry with the smallest timestamp and removes it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Batch and Streaming Log Deduplication
#
# Process (timestamp, message) logs by deduplicating on
# message and sorting retained entries by timestamp, first
# keeping the first-seen entry and then the latest
# timestamp. The streaming extension adds add(timestamp,
# message) and get_next(), requiring a map plus min-heap,
# lazy deletion, and a discussion of watermarks and
# bounded state.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Late data: how do you decide when it is safe to emit?
```

```
# 2. Memory must stay bounded.
# 3. Keep the entry with the most occurrences instead.
# 4. Only release entries once they are safe (late data
# may still arrive).
#
# INTERVIEW NOTES
# Dedup is a dictionary keyed by the message. Sorting is
# only for the output order. For a stream, a heap gives
# the smallest timestamp quickly, but when an entry is
# replaced you cannot delete from the middle of a heap, so
# leave the stale entry and skip it when it surfaces (lazy
# deletion).
#
# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates
# For each log line, scan the rest (or the earlier lines)
# to see if it is the one to keep. Clear logic, quadratic
# cost. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates | time O(n^2)
def dedupe_brute(logs, policy="first"):
    keep = []
    for i, (ts, msg) in enumerate(logs):
        if policy == "first":
            ok = all(
                logs[j][1] != msg for j in range(i)
            )
        else:
```

```
            ok = all(
                logs[j][1] != msg
                or logs[j][0] < ts
                or (logs[j][0] == ts and j < i)
                for j in range(len(logs))
                if j != i
            )
        if ok:
            keep.append((ts, msg))
    return sorted(keep, key=lambda e: (e[0], e[1]))


# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())


# FOLLOW-UP 4: Only release entries once they are safe
# (late data may still arrive).
# Answer: Add a watermark: release only entries whose
# timestamp is at most (the largest timestamp seen minus
# the allowed lateness), in order; anything older than the
# watermark that arrives later goes to a late-data path.
class StreamDedup:
```

```
    """add(ts, msg) and get_next() with a lazy heap."""

    def __init__(self, policy="first", lateness=0):
        self.policy, self.lateness = policy, lateness
        # message -> the timestamp currently kept
        self.kept = {}
```

```
        # (timestamp, message), may hold outdated entries
        self.heap = []
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        wm = self.watermark()
        # too late: older than the watermark, drop it
        if wm is not None and ts < wm:
            return False
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (
                self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        wm = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            # lazy deletion: skip entries that were
            # replaced
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            # not safe yet: later data may still arrive
            if ts > wm:
                return None
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None


if __name__ == "__main__":
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
```

```
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    # watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (1, "a")
    assert s.get_next() == (3, "c")
    # 5 is still above the watermark
    assert s.get_next() is None
    # data older than the watermark is dropped
    assert not s.add(2, "late")
    # a "latest" policy replaces the kept entry
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")
    assert t.get_next() is None
```

NOTES FROM CANDIDATES

Empty input, identical entries, equal timestamps, out-of-order arrivals, updates, and removals all need explicit semantics.
For keep-latest streaming behavior, stale heap entries must be detected against the current timestamp stored for that message.
Arbitrarily late arrivals make globally final timestamp ordering impossible without a watermark or maximum-lateness guarantee.

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

Easy (1)

EASY | 1Point3Acres | onsite-coding | 17 of 36

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 33 | PRIORITY 34/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

QUESTION

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

Logger API variant

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

EXAMPLES

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

APPROACH

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

HOW TO THINK ABOUT IT

The state that matters is the last time each message was displayed, not the last time it arrived. A message that is hidden must not move that time, otherwise a message repeated every few seconds would be hidden forever. A hash map from message to last displayed time answers each event in O(1).

Basis: LeetCode 359, Logger Rate Limiter (a message may be shown again only after the window has passed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

BRUTE FORCE

TIME	SPACE
O(displayed) per event	O(displayed)

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Robot Status Message Deduplication Window
#
# Process timestamped robot messages and display a message
# only when it has not been displayed within the prior 10
# seconds. Hidden arrivals do not refresh the last-
# displayed timestamp, and the exact 10-second boundary
# must be clarified. Process a stream of (time, message)
# status events from a robot. If the same message was
# displayed within the past 10 seconds, hide the new
# event. Otherwise display it and update that message's
# last-displayed time. Track the last time each message
# was displayed, not the last time it was received. A
# hidden duplicate must not refresh the window. Clarify
# whether an event exactly 10 seconds after the prior
# display is allowed: the comparison may be > 10 or >= 10.
# Logger API variant Implement
# shouldPrintMessage(timestamp, message), returning true
# when the message has not been printed in the past 10
# seconds and false otherwise.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events can arrive out of order.
#
# INTERVIEW NOTES
# The state that matters is the last time each message was
# displayed, not the last time it arrived. A message that
# is hidden must not move that time, otherwise a message
# repeated every few seconds would be hidden forever. A
# hash map from message to last displayed time answers
# each event in O(1).
#
# BRUTE FORCE (say it first): Keep every displayed time
# and scan them
# Store the list of every (time, message) shown so far and
# scan it for the same message within the window. Correct,
# but the scan grows with the whole history. Time
# O(displayed) per event. Space O(displayed).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Keep every displayed time
# and scan them | time O(displayed) per event
class DedupBrute:
    def __init__(self, window=10):
        self.window, self.shown = window, []

    def show(self, t, msg):
        if any(
            m == msg and t - s < self.window
            for s, m in self.shown
        ):
            return False
        self.shown.append((t, msg))
        return True


# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# FOLLOW-UP 1: Events can arrive out of order.
# Answer: Decide the contract first: either a late event
# is judged against the stored time with abs(t - last), or
# you keep a sorted history per message and compare with
# the nearest displayed neighbours on both sides.
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
```

```
            return False
        insort(times, t)
        return True


if __name__ == "__main__":
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")
```

NOTES FROM CANDIDATES

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

PRACHUB

8 questions | Hard 3, Medium 4, Easy 1 | posted in the last 30 days: 0, last 90 days: 8 (grouped by recency, newest first) | topics: BFS/DFS and trees first

POSTED: 31 TO 90 DAYS AGO (8)

Hard (3)

HARD | IN APP LC 1325 | PracHub | Technical Screen | 18 of 36

Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

POSTED 2026-09-16 | 34 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 10.6 30 DAYS 0 PRIORITY 164/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1325 Delete Leaves With a Given Value

LeetCode match: LC 1325 Delete Leaves With a Given Value

Posted on PracHub: 2026-09-16 (updated 2026-09-20)

Original: https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.

QUESTION

- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

APPROACH

Post-order DFS: process children first, then delete the node if it became a leaf with value == target. O(n).

HOW TO THINK ABOUT IT

'Repeatedly' hides an order: a node becomes deletable only after its children are processed. That is a post-order dependency, so a single bottom-up pass does what many passes would do. Whenever a rule says 'repeat until nothing changes' on a tree, ask whether children-first order collapses it to one pass.

Basis: LeetCode 1325, Delete Leaves With a Given Value.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def delete_pass(node):
    if node is None:
        return None, 0
    node.left, a = delete_pass(node.left)
    node.right, b = delete_pass(node.right)
    if is_leaf(node) and node.val == target:
        return None, a + b + 1
    return node, a + b
while True:
    root, removed = delete_pass(root)
    if removed == 0:
        break
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

BRUTE FORCE

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

OPTIMAL SOLUTION · One post-order pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

FOLLOW-UP 2 Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. **Cost:** one post-order pass, O(n) time and O(h) space.

FOLLOW-UP 3 Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. **Cost:** O(1) extra, counted inside the same O(n) pass.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Repeatedly Delete Leaves Matching a
# Target Value From a Binary Tree
#
# Given a binary tree and an integer target, delete every
# leaf whose value equals target. Deleting a leaf can turn
# its parent into a leaf; if that parent's value also
# equals target, it must be deleted as well, and so on,
# until no leaf with value target remains. Return the
# resulting tree, which is empty if the root itself ends
# up deleted. ### Function Signature
# remove_target_leaves(tree: list[int | None], target:
# int) -> list[int | None] ### Tree Encoding The input and
# the output use the same level-order encoding: - An empty
# tree is []. - Otherwise the first element is the root's
# value. Present nodes are then visited in breadth-first
# order, left to right, and for each one its left child
# and then its right child are appended, using None for an
# absent child. Children of absent nodes are not listed. -
# Trailing None values are omitted.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a 100,000-node chain. How do you avoid a
# stack overflow?
# 2. Delete any node whose whole subtree sums to zero, or
# whose value matches a predicate.
# 3. Return how many nodes were deleted.
#
# INTERVIEW NOTES
# 'Repeatedly' hides an order: a node becomes deletable
# only after its children are processed. That is a post-
# order dependency, so a single bottom-up pass does what
# many passes would do. Whenever a rule says 'repeat until
# nothing changes' on a tree, ask whether children-first
# order collapses it to one pass.
#
# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing
# Each pass deletes only the current leaves with the
# target value; a parent that becomes a matching leaf
# waits for the next pass. On a chain of n matching nodes
# that is n passes of O(n) each. Time O(n * h). Space
# O(h).


from collections import deque
```

```
# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing | time O(n * h)
def remove_leaves_brute(root, target):
    def one_pass(node):
        if node is None:
            return None, False
        if node.left is None and node.right is None:
            return (
                (None, True)
                if node.val == target
                else (node, False)
            )
        node.left, a = one_pass(node.left)
        node.right, b = one_pass(node.right)
        return node, a or b

    changed = True
    while changed:
        root, changed = one_pass(root)
    return root


# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node
```

```
    return dfs(root), deleted
```

```
# FOLLOW-UP 1: The tree is a 100,000-node chain. How do
# you avoid a stack overflow?
# Answer: Collect nodes in pre-order with an explicit
# stack and remember each node's parent, then walk that
# list in reverse (children before parents) and unlink
# matching leaves. This keeps the post-order guarantee
# without recursion.
def remove_leaves_iterative(root, target):
    # no recursion: safe for a 100,000-node chain
    if root is None:
        return None, 0
    parent, order, stack = {root: None}, [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    # reversed order: children are handled before parents
    for node in reversed(order):
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(values):
```

```
    # level-order list with None for missing children
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        # next two values are this node's children
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


def to_list(root):
    # the same encoding, with trailing Nones removed
    out, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        out.append(node.val if node else None)
        if node:
            queue.append(node.left)
            queue.append(node.right)
    while out and out[-1] is None:
        out.pop()
    return out
```

```
def remove_target_leaves(values, target):
    root, _ = remove_leaves(build(values), target)
    return to_list(root)


if __name__ == "__main__":
    tree = [1, 2, 3, 2, None, 2, 4]
    result = remove_target_leaves(tree, 2)
    assert result == [1, None, 3, None, 4]
    # the whole tree can disappear
    assert remove_target_leaves([1, 1, 1], 1) == []
    assert remove_target_leaves([], 5) == []
    root, count = remove_leaves(build(tree), 2)
    assert count == 3
```

```
    again = remove_leaves_iterative(build(tree), 2)
    assert again[1] == 3
    assert to_list(again[0]) == to_list(root)
    # a 100,000 node chain works without recursion
    chain = TreeNode(7)
    cur = chain
    for _ in range(99999):
        cur.left = TreeNode(7)
        cur = cur.left
    gone = remove_leaves_iterative(chain, 7)
    assert gone == (None, 100000)
```

FUNCTION AND TEST CASES

Signature

```
def remove_target_leaves(tree: list[int | None], target: i
nt) -> list[int | None]:
```

Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

Test cases

Input	Expected
([5, 7, 7, 7, 3, None, 7], 7)	[5, 7, None, None, 3]
([3, 3, 1], 3)	[3, None, 1]

HARD | IN APP LC 41 | PracHub | Technical Screen | 19 of 36

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 48 30 DAYS 0 PRIORITY 73/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

ABOUT THIS QUESTION

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

APPROACH

Cyclic placement: swap each value v in 1..n into index v-1 until settled, then the first index i where nums[i] != i+1 gives the answer (else n+1). O(n) time, O(1) space.

HOW TO THINK ABOUT IT

The answer is between 1 and n + 1, because n numbers can cover at most n of the positive integers. So the array itself can be used as the 'seen' table: put each value v (1 <= v <= n) at index v - 1. After that, the first index i where nums[i] != i + 1 gives the answer i + 1.

Basis: LeetCode 41, First Missing Positive.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Smallest Missing Positive Integer in
# Linear Time and Constant Extra Space
#
# You are given an unsorted list of integers nums, which
# may contain negative numbers, zeros, duplicates, and
# values far larger than the length of the list. Return
# the smallest positive integer, that is the smallest
# integer that is at least 1, that does not appear in
# nums. In the interview, a first solution that ran in
# linear time but used extra memory proportional to the
# input was accepted as a starting point, and the
# interviewer then asked for the same result using only
# constant extra space. Aim for that tighter target. ###
# Function Signature python def
# first_missing_positive(nums: list[int]) -> int: ###
# Rules - Target complexity: O(n) time and O(1) auxiliary
# space, where n = len(nums). The list may be rearranged
# or overwritten in place; its contents after the call do
# not matter. - Return only the missing value.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without swapping (sign marking).
# 2. The array must not be modified.
# 3. Smallest missing non-negative integer, or the k-th
# missing positive.
#
# INTERVIEW NOTES
# The answer is between 1 and n + 1, because n numbers can
# cover at most n of the positive integers. So the array
# itself can be used as the 'seen' table: put each value v
# (1 <= v <= n) at index v - 1. After that, the first
# index i where nums[i] != i + 1 gives the answer i + 1.
#
# BRUTE FORCE (say it first): Hash set of the values, then
# count up
# Put all numbers in a set and test 1, 2, 3, ... until one
# is missing. Linear time but linear extra memory, which
# is the version the interviewer wants to improve. Time
# O(n). Space O(n).


from bisect import bisect_left


# BASE QUESTION (optimal): Cyclic placement: put each
```

```
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# EASY VERSION: O(n) memory (say it first)
```

```
def first_missing_set(nums):
    # the easy O(n) memory answer (fine as a first
    # version)
    seen, v = set(nums), 1
    while v in seen:
        v += 1
    return v


# VARIANT: k-th missing positive
def kth_missing_positive(sorted_nums, k):
    # follow-up: k-th missing positive in a sorted array.
    # missing before index i is a[i] - (i + 1): binary
    # search
    lo, hi = 0, len(sorted_nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_nums[mid] - (mid + 1) < k:
            lo = mid + 1
        else:
            hi = mid
    return lo + k


# FOLLOW-UP 1: Do it without swapping (sign marking).
```

```
# Answer: Replace out-of-range values by n + 1, then for
# every value v in 1..n make the entry at index v - 1
# negative as a 'seen' flag; the first positive entry
# marks the missing number. Same O(n) time and O(1) space.
def first_missing_marking(nums):
    # no swaps: use the sign of a[v - 1] as a "seen" flag
    a, n = list(nums), len(nums)
    # values outside 1..n cannot matter: replace them
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    # the first positive entry marks the missing number
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1


if __name__ == "__main__":
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]
    for nums, want in cases:
        assert first_missing_positive(nums) == want
        assert first_missing_marking(nums) == want
        assert first_missing_set(nums) == want
    # the input list is left untouched
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]
    # 2, 3, 4, 7, 11: the 5th missing positive is 9
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6
```

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
`([4, -2, 1, 2],)`	`3`
`([5, 6, 100, -3],)`	`1`

HARD | PracHub | Technical Screen | 20 of 36

Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 145/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: the interview site's own solution

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.

QUESTION

- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **`R`** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

APPROACH

Per-cell DP over four directions (horizontal, vertical, diagonal, anti-diagonal): run[d][i][j] = run[d][prev]+1 if same color else 1. O(m*n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether diagonals count in both directions (yes, anti-diagonals too), whether cells must share an exact color value, and what to return for an empty grid. Confirm that a run cannot bend or skip a cell.

BRUTE FORCE TO SAY OUT LOUD

For every cell and each of the four directions, walk forward while the color matches. That is O(m * n * min(m, n)) and is a fine first answer to say out loud.

OPTIMAL IDEA

Let run[d][i][j] be the length of the same-color run that ends at (i, j) going in direction d. It is run[d][prev] + 1 if the previous cell in that direction has the same color, else 1. Four directions need only the previous row, so memory can be O(n). Answer is the maximum over everything. Time O(m * n).

DRY-RUN THESE CASES

A single cell (answer 1). A 1 x n row of one color. A grid where the longest run is on the anti-diagonal. A grid where every neighbor differs (answer 1).

LIKELY FOLLOW-UPS

Return the cells of the run, not just the length. Allow one wildcard cell. Make the run length at least K and count how many such runs exist.

PITFALLS

Forgetting the anti-diagonal needs the up-right previous cell, so it must be filled in the right scan order. Counting a bent path as a run.

HOW TO THINK ABOUT IT

A run in a fixed direction depends only on the cell before it in that direction, so each of the four directions is a one-line recurrence (like longest common suffix). Only four directions are needed because a line read backwards is the same line. Rolling only the previous row keeps memory at O(n).

Basis: Written; the same recurrence as LeetCode 562, Longest Line of Consecutive One in Matrix, with any color instead of 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION · DP per direction with a rolling row

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Longest Straight Run of One
# Color
#
# Given a rectangular grid of colors, return the length of
# the longest consecutive sequence of cells with the same
# color along a horizontal, vertical, or diagonal straight
# line. A sequence cannot change direction or skip a cell.
# Both diagonal orientations count. The length is the
# number of cells in the sequence. ### Input - grid: an
# array of strings of equal length. Each character
# represents one cell's color. ### Output Return the
# maximum length as an integer. ### Constraints and Edge
# Cases - For this practice version, the grid contains at
# least one row and one column, with at most 1000000 cells
# in total. - Colors are uppercase English letters; equal
# letters mean equal colors. - A one-cell grid has answer
# 1. - Diagonal lines may run down-right or down-left.
# Reading the same line in reverse does not change its
# length. - A connected region that bends is not
# necessarily one valid sequence.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the run is, not only its length.
# 2. Require the run to be at least K and count how many
# such runs exist.
# 3. The grid is huge and arrives row by row.
#
# INTERVIEW NOTES
# A run in a fixed direction depends only on the cell
# before it in that direction, so each of the four
# directions is a one-line recurrence (like longest common
# suffix). Only four directions are needed because a line
# read backwards is the same line. Rolling only the
# previous row keeps memory at O(n).
#
# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions
# For each start cell and direction, step while the color
# matches. Correct and short, but cells in a long run are
# re-walked from every start. Time O(m * n * min(m, n)).
# Space O(1).


# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions | time O(m * n * min(m, n))
```

```
DIRS4 = [(0, 1), (1, 0), (1, 1), (1, -1)]


def longest_run_brute(grid):
    best = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            for dr, dc in DIRS4:
                k = 0
                while (
                    0 <= r + dr * k < len(grid)
                    and 0 <= c + dc * k < len(grid[0])
                    and grid[r + dr * k][c + dc * k]
                    == grid[r][c]
```

```
                ):
                    k += 1
                best = max(best, k)
    return best


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)
```

```
# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))


# FOLLOW-UP 1: Return where the run is, not only its
# length.
# Answer: Only start a walk at cells that begin a run (the
# previous cell in that direction has a different color);
# that keeps the total work linear, and you record the
# start cell and direction of the best run.
def longest_run_where(grid):
    # the length, the start cell and the direction
    if not grid or not grid[0]:
        return 0, None, None
    return max(runs(grid), key=lambda x: x[0])


# FOLLOW-UP 2: Require the run to be at least K and count
# how many such runs exist.
# Answer: Count a run once, at its start, when its length
# is at least K (use the begins-a-run test above).
# Counting at every cell would count a long run many
# times.
def count_runs_at_least(grid, k):
    # each run is counted once, at its start
    if not grid or not grid[0]:
        return 0
    return sum(1 for n, _, _ in runs(grid) if n >= k)


# FOLLOW-UP 3: The grid is huge and arrives row by row.
# Answer: The rolling-row version already needs only the
# previous row, so it works on a stream of rows with O(n)
# memory.
def longest_run_rows(row_stream):
    # rows arrive one by one: keep only the previous row's
```

```
    # runs
    best, prev = 0, None
    h = v = d = a = None
    for row in row_stream:
        n = len(row)
        nh, nv = [1] * n, [1] * n
        nd, na = [1] * n, [1] * n
        for c in range(n):
            if c and row[c] == row[c - 1]:
                nh[c] = nh[c - 1] + 1
            if prev is not None:
                if row[c] == prev[c]:
                    nv[c] = v[c] + 1
                if c and row[c] == prev[c - 1]:
                    nd[c] = d[c - 1] + 1
                if c + 1 < n and row[c] == prev[c + 1]:
                    na[c] = a[c + 1] + 1
            best = max(best, nh[c], nv[c], nd[c], na[c])
        prev, h, v, d, a = row, nh, nv, nd, na
    return best


if __name__ == "__main__":
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run(g1) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0
    length, start, direction = longest_run_where(g1)
    assert length == 4
    # runs of length at least 3 are counted once each
    assert count_runs_at_least(g1, 3) == 4
    assert longest_run_rows(iter(g1)) == 4
```

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)	4
(['RBG', 'BGR', 'GRB'],)	3

ALSO REPORTED · 1 more

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

ALSO REPORTED · 1 more

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.
Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.
Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

Medium (4)

MEDIUM | PracHub | Technical Screen | 21 of 36

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 49 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 126/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

QUESTION

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer mean such as 5 is **`5/1`**.
- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the window into the removed largest values and the retained values. Keep their sizes correct and ensure no retained value is larger than a removed value.
```

APPROACH

Keep the window in two structures (top-k multiset and the rest) with running sums, using two heaps with lazy deletion or a balanced BST; mean = rest_sum / (w - k). O(n log w).

HOW TO THINK ABOUT IT

The mean needs two sums: the whole window and the sum of its K largest values. Keep the window in a sorted container; the K largest are its last K elements. When one value enters or leaves, only one element crosses the boundary between 'kept' and 'removed', so the sum of the K largest can be updated in O(1) after an O(log) search.

Basis: LeetCode 480 (Sliding Window Median) style order statistics; the mean of the window after dropping exactly K occurrences of the largest values, returned as reduced fractions.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sliding-Window Mean After Removing the
# Largest K Values
#
# For every contiguous window of a fixed size, remove the
# largest k values by multiplicity and compute the mean of
# the remaining values. Process sliding updates
# incrementally rather than sorting each complete window
# again. Implement trimmed_window_means(nums: int[],
# window: int, k: int) -> string[]. Return means in window
# order as reduced fractions numerator/denominator, using
# a positive denominator. This exact fraction encoding is
# an explicit practice representation that avoids cross-
# language floating-point rounding differences. ###
# Constraints & Assumptions - 1 <= len(nums) <= 200000, 1
# <= window <= len(nums), and 0 <= k < window. - Values
# range from -1,000,000 through 1,000,000. Use a
# sufficiently wide integer for window sums. - Remove
# exactly k occurrences, not k distinct values.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the update O(log w) in the worst case.
# 2. Remove the smallest K instead, or both ends (trimmed
# mean).
# 3. Duplicates on the boundary.
# 4. Trim the smallest K as well (a trimmed mean).
#
# INTERVIEW NOTES
# The mean needs two sums: the whole window and the sum of
# its K largest values. Keep the window in a sorted
# container; the K largest are its last K elements. When
# one value enters or leaves, only one element crosses the
# boundary between 'kept' and 'removed', so the sum of the
# K largest can be updated in O(1) after an O(log) search.
#
# BRUTE FORCE (say it first): Sort every window and sum
# the kept values
# For each window sort a copy, drop the largest k values,
# and compute the exact fraction. Correct and the oracle;
# every window is sorted from scratch. Time O(n * w log
# w). Space O(w).


from bisect import bisect_left, bisect_right
from fractions import Fraction
```

```
# HELPERS for the brute force
def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


def brute(nums, window, k, both=False):
    # sort every window: the check used by the demo
    out = []
    for i in range(len(nums) - window + 1):
        part = sorted(nums[i:i + window])
        part = part[k:-k] if both and k else part
        part = part[:len(part) - k] if not both else part
```

```
        mean = Fraction(sum(part), len(part))
        out.append(as_text(mean))
    return out


# HELPERS for the base question
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# mean).
# Answer: Maintain the sum of the K smallest alongside the
# K largest with the mirror-image boundary updates; the
# mean of the middle values is (total - top - bottom) / (w
# - 2K). Checked against sorting every window.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
```

```
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
```

```
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
    return out
```

```
if __name__ == "__main__":
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    fast = trimmed_window_means
    assert fast(nums, 3, 1) == brute(nums, 3, 1)
    assert fast(nums, 4, 2) == brute(nums, 4, 2)
    # a reduced fraction: (1 + 4) / 2 = 5/2
    assert fast([5, 1, 4], 3, 1) == ["5/2"]
    # duplicates at the boundary: remove exactly k copies
    dup = [3, 3, 3, 3]
    assert fast(dup, 3, 1) == ["3/1", "3/1"]
    # trimmed mean: drop k smallest and k largest
    both = fast(nums, 5, 1, True)
    assert both == brute(nums, 5, 1, True)
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k:
int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
([1, 9, 2, 8], 3, 1)	['3/2', '5/1']
([-1, 1, 3], 2, 0)	['0/1', '2/1']

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 22 of 36

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 167/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

QUESTION

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'],
    ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

APPROACH

Min-heap of released locker numbers plus a counter for the next never-used number; store pops the smallest, pickup pushes back. Track the last successful store separately. O(log n) per op.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what store and pickup return, what happens when a customer stores twice, when no locker is free, and whether numbers are reused smallest first.

BRUTE FORCE TO SAY OUT LOUD

Scan lockers 1..n for the first free one on every store: O(n) per operation.

OPTIMAL IDEA

Keep a min-heap of released locker numbers and a counter next_new for lockers never used. store: if the customer already has one return -1; take the heap minimum if the heap is not empty, otherwise next_new if it is at most n, else return -1; remember customer to locker in a dict. pickup: free the customer's locker by pushing it on the heap. O(log n) per operation, and memory depends on used lockers, not n.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Pickup then store reuses the smallest freed number. Double store by one customer. Store when full. Pickup with no locker.

LIKELY FOLLOW-UPS

n up to a billion (still fine because of the counter), thread safety with a lock, or lockers with sizes.

PITFALLS

Initializing the heap with all n numbers. Not removing the customer record on pickup.

HOW TO THINK ABOUT IT

'Smallest free number' with numbers coming back means a min-heap of returned numbers. Numbers that were never handed out do not need to be stored: keep a counter for the next unused number and use the heap first. That keeps memory proportional to the number of lockers in use, not to n.

Basis: Written; the smallest-available-number pattern (LeetCode 1845 Seat Reservation Manager, 2336 Smallest Number in Infinite Set).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	**O(n)**

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	**O(lockers released and not reused)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. **Cost:** O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement a Reusable Locker Allocation
# System
#
# ## Problem A locker bank has lockers numbered from 1
# through n. Process operations in order. store(customer)
# assigns the smallest available locker and returns its
# number, or -1 if that customer already has a locker or
# none is available. pickup(customer) releases and returns
# that customer's locker, or -1 if no locker is assigned.
# last() returns the locker number from the most recent
# successful store, or -1 if there has not been one.
# Return one result per operation. ### Constraints &
# Assumptions - 1 <= n <= 200,000. - There are at most
# 200,000 operations. - Customer IDs are non-empty
# strings. - A released locker becomes immediately
# available for reuse. ### Clarifications - Failed store
# and pickup operations do not change last(). - The
# allocation rule is always the smallest currently free
# locker number. - A customer can store again after a
# successful pickup.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are 10^9 lockers.
# 2. Many threads call store and pickup.
# 3. Lockers of different sizes.
#
# INTERVIEW NOTES
# 'Smallest free number' with numbers coming back means a
# min-heap of returned numbers. Numbers that were never
# handed out do not need to be stored: keep a counter for
# the next unused number and use the heap first. That
# keeps memory proportional to the number of lockers in
# use, not to n.
#
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one
# Keep an array of owners and scan from 1 for the first
# empty slot on store. Obviously correct, and each store
# can touch every locker. Time O(n) per operation. Space
# O(n).
```

```
import heapq
```

```
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one | time O(n) per operation
class LockersBrute:
    def __init__(self, n):
        self.owner, self.of = [None] * (n + 1), {}

    def store(self, customer):
        if customer in self.of:
            return -1
        for i in range(1, len(self.owner)):
            if self.owner[i] is None:
                self.owner[i], self.of[customer] = (
                    customer,
                    i,
                )
                return i
        return -1

    def pickup(self, customer):
        i = self.of.pop(customer, None)
        if i is None:
            return -1
        self.owner[i] = None
        return i


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
```

```
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
        return locker
```

```
    def last(self):
        # failed operations never change this
        return self.last_store


# FOLLOW-UP 2: Many threads call store and pickup.
# Answer: Protect the heap, the counter and the customer
# map with one lock so a locker number is never handed out
# twice (or fine-grained: a lock around the (heap,
# counter) pair).
class SafeLockers(Lockers):
    """Many threads: one lock around every operation."""

    def __init__(self, n):
        import threading
        super().__init__(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)
```

```
    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)


# HELPERS for the checks below
def process(n, operations):
    box, out = Lockers(n), []
    for op in operations:
        if op[0] == "store":
            out.append(box.store(op[1]))
        elif op[0] == "pickup":
            out.append(box.pickup(op[1]))
        else:
            out.append(box.last())
    return out


if __name__ == "__main__":
    ops = [["store", "a"], ["store", "b"],
           ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, ops) == [1, 2, 1, 1, 1]
    # nothing stored yet, and a missing pickup
    empty = [["last"], ["pickup", "x"]]
    assert process(2, empty) == [-1, -1]
    # full, and a customer who already has a locker
    full = [["store", "a"], ["store", "a"],
            ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]
    # failed operations leave last() alone
    more = full + [["last"]]
    assert process(2, more)[-1] == 2
    # a huge locker bank costs nothing up front
    one = [["store", "a"], ["last"]]
    assert process(10 ** 9, one) == [1, 1]
    s = SafeLockers(2)
    assert s.store("a") == 1 and s.pickup("a") == 1
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[lis
t[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b
'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

MEDIUM | PracHub | Technical Screen | 23 of 36

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 72 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 0 | PRIORITY 83/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

APPROACH

DFS/backtracking from every start vertex, extending a path only while the labels match the next query character and the vertex is unvisited (visited bitmask); count paths for every prefix length.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the graph is directed, whether a path is counted as a vertex sequence, whether paths of every prefix length are counted (the statement says every nonempty prefix), and the size (small, since the state is a visited subset).

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths and then test labels. Exponential and wasteful.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

DFS with backtracking from every start vertex. Extend a path only if the next vertex is unvisited (a bitmask of visited vertices) and its label equals the next character of the query word. Every time the path length is L, count one for prefix length L. The pruning on labels keeps the search small. Time depends on the number of matching paths, bounded by n * d^(L-1).

DRY-RUN THESE CASES

A query of length 1 (count of vertices with that label), a graph with a cycle (simple paths may not revisit), a label that appears nowhere, repeated letters in the query.

LIKELY FOLLOW-UPS

Longer queries (memoize on (vertex, mask, position) only helps if counting, not listing), or undirected edges.

PITFALLS

Counting a path that revisits a vertex. Counting an undirected path twice (once in each direction) without agreeing on it.

HOW TO THINK ABOUT IT

A path is only worth extending while its labels still match the query, so prune on the label before going deeper. Distinctness is tracked with a visited set (a bitmask for small graphs). Count at every depth, since each prefix length has its own answer.

Basis: Written; backtracking on a small labelled graph. Assumed rules (confirm them): directed edges; a path is a sequence of distinct vertices; for each L from 1 to the length of the query, count the paths of L vertices whose labels spell the first L letters of the query.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Prefix-Matching Simple Paths in a
# Labeled Graph
#
# Count distinct simple graph paths whose labels match
# every nonempty prefix of a query word. Paths may start
# anywhere but cannot revisit a vertex, so the task
# rewards subset-state search and careful counting over a
# small labeled graph.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges are undirected.
# 2. Count only the paths that spell the whole query.
# 3. The graph has up to 100,000 vertices.
#
# INTERVIEW NOTES
# A path is only worth extending while its labels still
# match the query, so prune on the label before going
# deeper. Distinctness is tracked with a visited set (a
# bitmask for small graphs). Count at every depth, since
# each prefix length has its own answer.
#
# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels
# List every simple path of every length and compare its
# labels with each prefix of the query. Correct and the
# oracle; it explores paths whose first label already
# disagrees. Time exponential. Space O(path).
```

```
from collections import defaultdict


# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels | time exponential
def prefix_paths_brute(labels, edges, query):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    counts = [0] * len(query)

    def go(path):
        word = "".join(labels[v] for v in path)
        if (
            len(path) <= len(query)
            and word == query[: len(path)]
```

```
        ):
            counts[len(path) - 1] += 1
        if len(path) < len(query):
            for w in adj[path[-1]]:
                if w not in path:
                    go(path + [w])

    for s in range(len(labels)):
        go([s])
    return counts


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
```

```
            go(s, 0, 1 << s)
    return counts


# VARIANT: walks may revisit vertices
def prefix_walks(labels, edges, query):
    # walks may revisit vertices: a DP over (depth,
    # vertex)
    # costs O(L * E) and works for very large graphs
    adj = defaultdict(list)
```

```
    for u, v in edges:
        adj[u].append(v)
    ways = {v: 1 for v in range(len(labels))
            if query and labels[v] == query[0]}
    counts = [sum(ways.values())] if query else []
    for ch in query[1:]:
        nxt = defaultdict(int)
        for u, n in ways.items():
            for w in adj[u]:
                if labels[w] == ch:
                    nxt[w] += n
        ways = nxt
        counts.append(sum(ways.values()))
    return counts


# FOLLOW-UP 2: Count only the paths that spell the whole
# query.
# Answer: Return counts[-1] instead of the list. For long
# queries on bigger graphs, memoise on (vertex, depth) if
# repeated vertices are allowed (walks, not simple paths);
# for simple paths the visited set prevents memoisation.
def whole_query_paths(labels, edges, query,
                      undirected=False):
    # only the paths that spell the entire query
    both = undirected
    counts = prefix_paths(labels, edges, query, both)
    return counts[-1] if counts else 0


if __name__ == "__main__":
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # "aba": directed square a>b>a>b>...
    assert prefix_paths(labels, edges, "aba") == [2, 2, 2]
```

```
    assert whole_query_paths(labels, edges, "aba") == 2
    # undirected: every edge works both ways
    und = prefix_paths(labels, edges, "ab", True)
    assert und == [2, 4]
    # a walk may reuse vertices, a simple path may not
    walks = prefix_walks(labels, edges, "abab")
    assert walks == [2, 2, 2, 2]
    five = prefix_paths(labels, edges, "ababa")
    assert five == [2, 2, 2, 2, 0]
    assert prefix_paths(labels, edges, "") == []
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`

MEDIUM | PracHub | Technical Screen | 24 of 36

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 46/228

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

APPROACH

DFS expansion with memoization per key and a 'visiting' set to detect cycles; leave unknown keys per the spec; keep literal text unchanged.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the placeholder syntax (%KEY%), what to do with an unknown key (leave it as is or raise), how to write a literal percent sign (escaping such as %%), and how to handle cycles (raise an error).

BRUTE FORCE TO SAY OUT LOUD

Repeatedly run a regex replace on the whole string until nothing changes. It loops forever on a cycle.

OPTIMAL IDEA

Resolve each key with memoized DFS: resolve(key) expands the value of that key by scanning for placeholders and recursively resolving them, with a visiting set to detect cycles and a cache so each key is expanded once. Expand the template by scanning once and replacing each placeholder with resolve(name), leaving literal text untouched. Linear in the sizes of text and values.

DRY-RUN THESE CASES

A value that refers to another key, a cycle (A refers to B, B to A), an unknown key, a template with no placeholders, two placeholders adjacent, an escaped percent.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Escaping, recursion depth limit (iterative expansion), or limiting the output size to prevent blow-up.

PITFALLS

Re-expanding a key every time. Treating an unknown key and a cycle the same way.

HOW TO THINK ABOUT IT

Expanding one key can require expanding others, so define expand(key) recursively and remember results (memoization) so a key used many times is expanded once. A 'currently expanding' set detects cycles: meeting a key that is already on the stack means a cycle. Decide the policy for unknown keys and for literal percent signs up front.

Basis: Written. Assumed rules (confirm them): placeholders look like %KEY%; the value of a key may itself contain placeholders; an unknown key is left as is; a cycle raises an error.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Expand Recursive Key-Value Placeholders
# in a Template
#
# Expand %KEY% placeholders in a template, recursively
# resolving placeholders inside mapped values. Use
# memoized dependency traversal to preserve literal text,
# avoid repeated work, and explain cycle, unknown-key, and
# escaping policies.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A literal percent sign must be possible.
# 2. An unknown key should be an error, or replaced by an
# empty string.
# 3. Values are huge and expansion explodes.
#
# INTERVIEW NOTES
# Expanding one key can require expanding others, so
# define expand(key) recursively and remember results
# (memoization) so a key used many times is expanded once.
# A 'currently expanding' set detects cycles: meeting a
# key that is already on the stack means a cycle. Decide
# the policy for unknown keys and for literal percent
# signs up front.
#
# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing
# Replace every %KEY% found in the text by its value and
# repeat until nothing changes. Simple, but it never stops
# on a cycle (so it needs an iteration cap) and re-expands
# the same key every time it appears. Time O(depth *
# length). Space O(length).


import re


# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing | time O(depth * length)
_PH = re.compile(r"%([A-Za-z_][A-Za-z0-9_]*)%")
```

```
# HELPERS for the base question
PATTERN = re.compile(r"%%|%(\w+)%")
```

```
def expand_by_rewriting(
    template, mapping, max_rounds=50
):
    text = template
    for _ in range(max_rounds):
        new = _PH.sub(
            lambda m: mapping.get(
                m.group(1), m.group(0)
            ),
            text,
        )
        if new == text:
            return text
        text = new
    raise ValueError("cycle")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
# FOLLOW-UP 1: A literal percent sign must be possible.
# Answer: Choose an escape such as %% and handle it in the
# same scan: when the text has %%, output one % and skip
# both characters; otherwise try to match a placeholder.
# Test percent signs next to placeholders.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
```

```
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)
```

```
if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    want = "I am admin My home is /admin/home"
    assert expand(text, m) == want
    # unknown keys: kept, emptied, or an error
    assert expand("a %X% b", m) == "a %X% b"
    assert expand("a %X% b", m, unknown="empty") == "a  b"
    try:
        expand("%X%", m, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass
    # a literal percent, even next to a placeholder
    assert expand("100%% of %USER%", m) == "100% of admin"
    # a cycle is reported
    try:
        expand("%A%", {"A": "%B%", "B": "%A%"})
        raise AssertionError("cycle not caught")
    except ValueError:
```

```
        pass
    # a key that doubles the previous one explodes
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom[f"K{i}"] = f"%K{i - 1}%%K{i - 1}%"
    try:
        expand("%K29%", boom, limit=1000)
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

ALSO REPORTED · 1 more

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

Easy (1)

EASY | IN APP LC 729 | PracHub | Technical Screen | 25 of 36

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 26.5 30 DAYS 0 PRIORITY 132/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

APPROACH

Half-open intervals [s,e) conflict iff new.start < e and s < new.end (touching endpoints do not conflict). One-off check on an unsorted collection is a single O(n) scan; for repeated queries keep a sorted structure (TreeMap/balanced BST or sorted list + bisect) and check only the neighbors. O(log n) per query.

HOW TO THINK ABOUT IT

Two half-open intervals [s1, e1) and [s2, e2) overlap exactly when s1 < e2 and s2 < e1. If the booked intervals are kept sorted and disjoint, only the neighbours at the insertion point can conflict, so a binary search replaces the scan.

Basis: LeetCode 729, My Calendar I (booking succeeds only when the new half-open interval overlaps nothing), and 731 for double booking.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check an Interval Against Existing
# Intervals for Conflicts
#
```

```
# Check whether one half-open candidate interval conflicts
# with any interval in a large unsorted collection.
# Respect strict start and end semantics so ranges that
# only touch at an endpoint do not conflict.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Allow double booking but never triple booking
# (LeetCode 731).
# 2. Report the maximum number of overlapping bookings
# after each booking (LeetCode 732).
# 3. Closed intervals instead of half-open.
#
# INTERVIEW NOTES
# Two half-open intervals [s1, e1) and [s2, e2) overlap
# exactly when s1 < e2 and s2 < e1. If the booked
# intervals are kept sorted and disjoint, only the
# neighbours at the insertion point can conflict, so a
# binary search replaces the scan.
#
# BRUTE FORCE (say it first): Scan every booked interval
# Compare the new interval with all existing ones using
# the overlap test. Correct and simple, but each booking
# costs time proportional to the number of bookings. Time
# O(n) per booking. Space O(n).


from bisect import bisect_right
from collections import defaultdict


# BRUTE FORCE (say it first): Scan every booked interval |
# time O(n) per booking
class CalendarBrute:
    def __init__(self):
        self.booked = []

    def book(self, start, end):
        if any(
            start < e and s < end
            for s, e in self.booked
        ):
            return False
```

```
        self.booked.append((start, end))
        return True


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# FOLLOW-UP 1: Allow double booking but never triple
# booking (LeetCode 731).
# Answer: Keep a difference map (+1 at start, -1 at end)
# and sweep it in sorted order; if the running count ever
# exceeds 2, undo the change and reject. O(n log n) per
# booking, or use a segment tree for faster.
class DoubleBooking:
    """LeetCode 731: two overlaps are fine, three are not.
    LeetCode 732: also report the largest overlap."""

    def __init__(self, limit=2):
        self.delta = defaultdict(int)
        self.limit = limit

    def peak(self):
        # sweep the +1 / -1 changes in time order
        run = best = 0
        for t in sorted(self.delta):
            run += self.delta[t]
```

```
            best = max(best, run)
        return best

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        if self.peak() > self.limit:
            # undo the change and reject the booking
            self.delta[start] -= 1
            self.delta[end] += 1
            return False
        return True


# HELPERS for the checks below
def conflicts(candidate, intervals):
    # half-open [start, end): touching ends do NOT
    # conflict
    s, e = candidate
    return any(a < e and s < b for a, b in intervals)


if __name__ == "__main__":
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    # sorted calendar
    c = Calendar()
    assert c.book(10, 20) and c.book(20, 30)
    assert not c.book(15, 25) and c.book(5, 10)
    # double booking is allowed, triple booking is not
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)
    assert not d.book(17, 18)
    assert d.peak() == 2
    # limit 1 is the plain no-overlap calendar
    one = DoubleBooking(limit=1)
    assert one.book(1, 5) and not one.book(4, 6)
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: l
ist[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
`([4,7],[[1,3],[7,9]])`	`False`
`([4,7],[[1,5],[8,9]])`	`True`

EXPONENT

11 questions | Hard 2, Medium 8, Easy 1 | last asked (approx) in the last 30 days: 0, last 90 days: 11 (grouped by recency, newest first) | topics: BFS/DFS and trees first

ASKED (ROUGH): 31 TO 90 DAYS AGO (11)

Hard (2)

HARD | IN APP LC 146 | DESIGN-STYLE (skippable) | Exponent | 26 of 36

★ TOP 75: rank 64 in the Medium and Hard priority list

★ Implement LRU Cache.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.3 | 30 DAYS 0 | PRIORITY 48/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #146)

LeetCode match: LC 146 LRU Cache

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +2562 interviews

Original: https://www.aced.io/questions/3625/lru-cache

QUESTION

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time? Pythonclass LRUCache(n): set(key, value) get(key) Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated. Pythoncache = LRUCache(2)

Limit of 2 items cache.set('user1',

'Alex') cache.set('user2', 'Brian') cache.set('user3', 'Chris') Here user1 is empty because it was the least recently used and thus removed to make room for user3: Pythoncache.get('user1')

=> None cache.get('user2')

=> 'Brian' cache.get('user3')

=> 'Chris'Hint 1If you're having

QUESTION

trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array

APPROACH

Hash map + doubly linked list (or OrderedDict): get/put move the node to the front, evict from the tail when over capacity. O(1) per op.

HOW TO THINK ABOUT IT

You need two things at once: find a key quickly (a hash map) and know which key is the oldest use (an ordering you can update in O(1)). A doubly linked list gives O(1) move-to-front and remove-from-back, and each hash map entry points at its list node. Python's OrderedDict is exactly that pair; in an interview be ready to build the linked list by hand.

Basis: LeetCode 146, LRU Cache: get and put in O(1), evicting the least recently used key when over capacity.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []  # (key, value), newest last
def get(k):
    for i, (key, value) in enumerate(items):
        if key == k:
            items.append(items.pop(i))  # newest
            return value
    return -1
def put(k, v):
    for i, (key, _) in enumerate(items):
        if key == k:
            items.pop(i)
            break
    else:
        if len(items) == capacity:
            items.pop(0)  # oldest
    items.append((k, v))
```

TIME	SPACE
O(n) per operation	O(capacity)

WHY IT IS SLOW

Every operation scans the list, O(capacity). A hash map to nodes of a doubly linked list gives O(1) lookup and O(1) move-to-front and eviction.

OPTIMAL SOLUTION · Hash map plus doubly linked list (built by hand)

TIME	SPACE
O(1) per operation	O(capacity)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many threads share the cache.

Wrap get and put in one lock (both reorder the list, so even get is a write). For more throughput shard the cache by key hash, each shard with its own lock and capacity.

FOLLOW-UP 2 Entries expire after a time to live (TTL).

Store an expiry time with each entry, check it on get (a stale entry is treated as missing and removed), and optionally sweep from the old end of the list on put. The recency order and the O(1) operations are unchanged.

FOLLOW-UP 3 Least frequently used (LFU) instead of least recently used.

Keep a frequency per key and buckets of keys per frequency (each bucket an ordered dict) with a pointer to the minimum frequency; get and put move a key to the next bucket in O(1) (LeetCode 460).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement LRU Cache.
#
# Your program is running slowly because it's accessing
# data from disk over and over again. To improve the
# performance, you want to build a simple key-value store
# to cache this data in memory, but you also want to limit
# the amount of memory used. You decide to build a caching
# system that only keeps the N most recently used
# items—also known as a least recently used (LRU) cache.
# Write a class LRUCache(n) that accepts a size limit n.
# It should support a set(key, value) method for inserting
# or updating items and a get(key) method for retrieving
# items. Can you implement a solution where both of these
# methods run in O(1) time? Pythonclass LRUCache(n):
# set(key, value) get(key) Examples To make room for new
# items, the least recently used item should be removed.
# An item is 'used' whenever it is set, retrieved, or
# updated.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many threads share the cache.
# 2. Entries expire after a time to live (TTL).
# 3. Least frequently used (LFU) instead of least recently
# used.
#
# INTERVIEW NOTES
# You need two things at once: find a key quickly (a hash
# map) and know which key is the oldest use (an ordering
# you can update in O(1)). A doubly linked list gives O(1)
# move-to-front and remove-from-back, and each hash map
# entry points at its list node. Python's OrderedDict is
# exactly that pair; in an interview be ready to build the
# linked list by hand.
#
# BRUTE FORCE (say it first): A list of keys ordered by
# recency
# Keep a list of (key, value) with the most recent at the
# end. get and put search the list and move the entry.
# Obviously correct and the oracle; every operation is a
# linear scan. Time O(n) per operation. Space O(capacity).


import threading
import time
from collections import OrderedDict, defaultdict


# BRUTE FORCE (say it first): A list of keys ordered by
# recency | time O(n) per operation
class LRUBrute:
    def __init__(self, capacity):
        self.cap, self.items = capacity, []

    def get(self, key):
        for i, (k, v) in enumerate(self.items):
            if k == key:
                self.items.append(self.items.pop(i))
                return v
        return -1

    def put(self, key, value):
        for i, (k, _) in enumerate(self.items):
            if k == key:
                self.items.pop(i)
                break
        else:
            if len(self.items) == self.cap:
                self.items.pop(0)
        self.items.append((key, value))


# HELPERS for the base question
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = None


# BASE QUESTION (optimal): Hash map plus doubly linked
# list (built by hand)
# Idea: Each key maps to a node of a doubly linked list
# ordered by recency, with sentinel head and tail nodes so
# no operation has to special-case an empty list.
class LRUCache:
    """get and put in O(1): a hash map plus a doubly
    linked list, most recently used item at the front."""

    def __init__(self, capacity):
        self.cap, self.map = capacity, {}
        # sentinel nodes mean no special cases for the
        # ends
        self.head, self.tail = Node(), Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
```

```
        node.next.prev = node.prev

    def push_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.map.get(key)
        if node is None:
            return -1
        # reading makes it the most recently used
        self.unlink(node)
        self.push_front(node)
        return node.value

    def put(self, key, value):
        node = self.map.get(key)
        if node:
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.map) == self.cap:
            # full: evict the least recently used (at the
            # back)
            oldest = self.tail.prev
            self.unlink(oldest)
            del self.map[oldest.key]
        node = Node(key, value)
        self.map[key] = node
        self.push_front(node)


# FOLLOW-UP 1: Many threads share the cache.
# Answer: Wrap get and put in one lock (both reorder the
```

```
# list, so even get is a write). For more throughput shard
# the cache by key hash, each shard with its own lock and
# capacity.
class SafeLRU(LRUCache):
    """Many threads: even get edits the list: one lock."""

    def __init__(self, capacity):
        super().__init__(capacity)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return super().get(key)

    def put(self, key, value):
        with self.lock:
            super().put(key, value)


# FOLLOW-UP 2: Entries expire after a time to live (TTL).
# Answer: Store an expiry time with each entry, check it
# on get (a stale entry is treated as missing and
# removed), and optionally sweep from the old end of the
# list on put. The recency order and the O(1) operations
# are unchanged.
class TTLCache(LRUCache):
    """Entries also expire after ttl seconds."""

    def __init__(self, capacity, ttl,
                 clock=time.monotonic):
        super().__init__(capacity)
        self.ttl, self.clock, self.expiry = ttl, clock, {}

    def get(self, key):
        # a stale entry counts as missing and is removed
        stale = key in self.map and (
```

```
            self.expiry[key] <= self.clock())
        if stale:
            self.unlink(self.map.pop(key))
            del self.expiry[key]
        return super().get(key)

    def put(self, key, value):
        super().put(key, value)
        self.expiry[key] = self.clock() + self.ttl
```

```
        # drop expiry records of evicted keys
        for k in list(self.expiry):
            if k not in self.map:
                del self.expiry[k]


# FOLLOW-UP 3: Least frequently used (LFU) instead of
# least recently used.
# Answer: Keep a frequency per key and buckets of keys per
# frequency (each bucket an ordered dict) with a pointer
# to the minimum frequency; get and put move a key to the
# next bucket in O(1) (LeetCode 460).
class LFUCache:
    """Least FREQUENTLY used (LeetCode 460), O(1)."""

    def __init__(self, capacity):
        self.cap = capacity
        self.value, self.freq = {}, {}
        # frequency -> keys in that bucket, oldest first
        self.bucket = defaultdict(OrderedDict)
        self.low = 0

    def touch(self, key):
        f = self.freq[key]
        del self.bucket[f][key]
        # if this was the lowest bucket and it is empty,
        # move up
        if not self.bucket[f] and self.low == f:
            self.low += 1
        self.freq[key] = f + 1
        self.bucket[f + 1][key] = None

    def get(self, key):
        if key not in self.value:
            return -1
        self.touch(key)
        return self.value[key]

    def put(self, key, value):
        if self.cap == 0:
            return
        if key in self.value:
            self.value[key] = value
            self.touch(key)
            return
```

```
        if len(self.value) == self.cap:
            # evict the oldest key of the lowest frequency
            bucket = self.bucket[self.low]
            old, _ = bucket.popitem(last=False)
            del self.value[old], self.freq[old]
        self.value[key], self.freq[key] = value, 1
        self.bucket[1][key] = None
        self.low = 1


if __name__ == "__main__":
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == 1
    # 2 is now the least recently used, so 3 evicts it
    c.put(3, 3)
```

```
assert c.get(2) == -1 and c.get(3) == 3
c.put(1, 10)
assert c.get(1) == 10
s = SafeLRU(1)
s.put("a", 1)
s.put("b", 2)
assert s.get("a") == -1 and s.get("b") == 2
now = [0]
t = TTLCache(2, ttl=10, clock=lambda: now[0])
t.put("x", 1)
assert t.get("x") == 1
now[0] = 11
assert t.get("x") == -1
f = LFUCache(2)
f.put(1, 1)
f.put(2, 2)
assert f.get(1) == 1
# 2 was used least often, so it goes
f.put(3, 3)
assert f.get(2) == -1 and f.get(3) == 3
```

OFFICIAL PAGE TEXT

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time?

```
Pythonclass LRUCache(n):
  set(key, value)
  get(key)
```

Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated.

```
Pythoncache = LRUCache(2) # Limit of 2 items

cache.set('user1', 'Alex')
cache.set('user2', 'Brian')
cache.set('user3', 'Chris')
```

Here user1 is empty because it was the least recently used and thus removed to make room for user3:

```
Pythoncache.get('user1') # => None
cache.get('user2') # => 'Brian'
cache.get('user3') # => 'Chris'
```

OFFICIAL PAGE TEXT

Hint 1If you're having trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array won't work because moving items around takes O(n) time. Could you combine two different data structures to maintain this order and also get fast lookups?Hint 3You can use a doubly linked list to store the items in the correct order. When an item is used, just move its node to the front of the list. The least recently used node will always end up at the end of the list.Hint 4To look up items without traversing the entire list, you can also add a hash table that maps from the key to the item's node.Challenge 1Whats the time complexity of your solution?This problem is challenging because of the combination of constraints such as key-value lookup, size limits, and time complexity. Ultimately, there is no single data structure that fulfills all of these needs, so we have to get creative! Our solution is to implement the cache class with a linked list and a hash table. The linked list is doubly-linked, which allows us to insert and remove items in constant time without traversing the entire list. Every time an item is set, updated, or retrieved, we just move its Node to the front of the list. When the cache reaches its size limit, we can easily make room by removing the item at the end of the list. The linked list gives us O(1) insertion time, but by itself it has O(n) lookup time because we have to traverse the entire list to find a particular item. That's not ideal since we want to be able to look up an item quickly using its key. That's where the hash table comes in: we can use it add a mapping from an item's key to its Node. Together, these data structures give us a time complexity of O(1) for insertion, deletion, and retrieval in the worst case, and a space footprint of O(n). No answers contributed by the community yet.

HARD | IN APP LC 23 | Exponent | 27 of 36

Merge k sorted linked lists.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 49.4 30 DAYS 0 PRIORITY 71/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #23)

LeetCode match: LC 23 Merge k Sorted Lists

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +816 interviews

Original: https://www.aced.io/questions/1167/merge-k-sorted-linked-lists

QUESTION

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.Interview experiences6 shared‹›Revolut · 2 months ago"Its good, but I was bad and dont really remember much. But all rounds was good. HR was nice. Tech guy on tech interview was nice too."Software Engineer · Entry Level / L3FFloQast · 2 months ago"Screening and the system design with

director are virtual, while on-site loop in the office. The process is well organi

APPROACH

Min-heap of the k list heads (or divide and conquer pairwise merging). O(N log k).

HOW TO THINK ABOUT IT

At any moment the next output node is the smallest among the current heads of the k lists, which is a min-heap of size k. Each node is pushed and popped once, so the total cost is N log k. A second way is to merge lists in pairs, halving the number of lists each round, with the same bound.

Basis: LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(lists):
    values = []
    for head in lists:
        while head:
            values.append(head.val)
            head = head.next
    values.sort()
    dummy = tail = ListNode(0)
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next
```

BRUTE FORCE

TIME	SPACE
O(N log N)	O(N)

WHY IT IS SLOW

O(N log N) time and O(N) extra memory, ignoring that each list is already sorted. A min-heap of the k current heads is O(N log k) with O(k) memory.

OPTIMAL SOLUTION · Min-heap of the current heads

TIME	SPACE
O(N log k)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Merge the lists in pairs instead of using a heap.

Repeatedly merge neighbouring pairs with the two-list merge; there are log k rounds, each touching all N nodes: O(N log k) time and O(1) extra space (no heap).

FOLLOW-UP 2 The lists are too large for memory (files or streams).

The heap approach already works on streams: it holds one item per list. Read each input through an iterator and write the output as you go; this is the merge phase of an external sort (heapq.merge does it). **Cost:** O(N log k) time and O(k) memory; one sequential read per list.

FOLLOW-UP 3 Why not merge the lists one by one into a running result?

Merging list i into a result of size about i * (N / k) costs O(i * N / k), so the total is O(k * N), far worse than O(N log k).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Merge k sorted linked lists.
#
# Merge K Sorted Linked Lists. You are given an array of k
# linked-lists lists, each linked-list is sorted in
# ascending order. Your task is to merge all the linked-
# lists into one sorted linked-list and return it.
# Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each
# array represents a linked-list, the merged and sorted
# linked-list would be [1,1,2,3,4,4,5,6]. The output for
# the merged list should be in array form for simplicity.
# If lists = [], indicating an array of no linked-lists,
# the output should be []. If lists = [[]], representing
# an array with a single empty linked-list, the output
# should be []. If lists = [[],[1]], where the first
# linked-list is empty and the second contains a single
# element, the output should be [1]. If lists =
# [[-1,5,11],[],[6,10]], the merged and sorted linked-list
```

```
# should be [-1,5,6,10,11].
#
# FOLLOW-UPS (answered below, in this order):
# 1. Merge the lists in pairs instead of using a heap.
# 2. The lists are too large for memory (files or
# streams).
# 3. Why not merge the lists one by one into a running
# result?
#
# INTERVIEW NOTES
# At any moment the next output node is the smallest among
# the current heads of the k lists, which is a min-heap of
# size k. Each node is pushed and popped once, so the
# total cost is N log k. A second way is to merge lists in
# pairs, halving the number of lists each round, with the
# same bound.
#
# BRUTE FORCE (say it first): Collect all values, sort
# them, rebuild a list
# Read every node into one array, sort it and build a new
# list. Correct and the oracle; it ignores that each input
# is already sorted and uses O(N) extra memory. Time O(N
# log N). Space O(N).


import heapq
```

```
# HELPERS for the brute force
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


# BRUTE FORCE (say it first): Collect all values, sort
# them, rebuild a list | time O(N log N)
def _from_list(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head


# HELPERS for the base question
def _to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out


def merge_by_sorting(lists):
    vals = sorted(
        v for head in lists for v in _to_list(head)
    )
    return _from_list(vals)


# BASE QUESTION (optimal): Min-heap of the current heads
# Idea: Push the head of each non-empty list as (value,
# list index, node). Pop the smallest, append it to the
# output, and push its successor. The list index is a tie-
# breaker so nodes are never compared.
def merge_k_heap(lists):
    # a heap holds the current head of every list; the
    # node
    # index breaks ties so nodes are never compared
    heap = [(n.val, i, n)
            for i, n in enumerate(lists) if n]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            nxt = node.next
            heapq.heappush(heap, (nxt.val, i, nxt))
    return dummy.next


# HELPER: merge two lists
def merge_two(a, b):
    dummy = tail = ListNode()
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next


# FOLLOW-UP 1: Merge the lists in pairs instead of using a
# heap.
# Answer: Repeatedly merge neighbouring pairs with the
# two-list merge; there are log k rounds, each touching
# all N nodes: O(N log k) time and O(1) extra space (no
# heap).
def merge_k_pairs(lists):
    # no heap: merge neighbours in pairs, log k rounds,
    # O(1) space
    lists = [n for n in lists if n]
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            if i + 1 < len(lists):
                pair = (lists[i], lists[i + 1])
                merged.append(merge_two(*pair))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]
```

```
# FOLLOW-UP 2: The lists are too large for memory (files
# or streams).
# Answer: The heap approach already works on streams: it
# holds one item per list. Read each input through an
# iterator and write the output as you go; this is the
# merge phase of an external sort (heapq.merge does it).
def merge_streams(iterators):
    # lists too big for memory: stream them (heapq.merge
    # keeps
    # one item per input), the merge step of an external
    # sort
    return heapq.merge(*iterators)


# HELPERS for the checks below
def make(values):
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
```

```
    return out


if __name__ == "__main__":
    data = [[1, 4, 5], [1, 3, 4], [2, 6]]
    want = [1, 1, 2, 3, 4, 4, 5, 6]
    got = to_list(merge_k_heap([make(v) for v in data]))
    assert got == want
    got = to_list(merge_k_pairs([make(v) for v in data]))
    assert got == want
    assert merge_k_heap([]) is None
    assert merge_k_pairs([None]) is None
    assert to_list(merge_k_heap([None, make([1])])) == [1]
    got = list(merge_streams(iter(v) for v in data))
    assert got == want
    got = to_list(merge_two(make([1, 3]), make([2])))
```

```
    assert got == [1, 2, 3]
```

OFFICIAL PAGE TEXT

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.

Medium (8)

MEDIUM | IN APP LC 207 | Exponent | 28 of 36

★ TOP 75: rank 40 in the Medium and Hard priority list

★ Course Schedule

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE **ALL TIME 51.9** **30 DAYS 33** **PRIORITY 31/228**

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1017 interviews

Original: https://www.aced.io/questions/3313/course-schedule

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

APPROACH

Kahn's topological sort (or DFS cycle detection); the answer is true iff every course is processed. O(V+E).

HOW TO THINK ABOUT IT

Finishing every course is possible exactly when the prerequisite graph has no cycle. Kahn's algorithm tests that by repeatedly taking courses with no unmet prerequisites; if you can take all of them there is no cycle, and the order you took them is a valid schedule.

Basis: LeetCode 207 (can you finish all courses) and 210 (return an order). A pair (a, b) means b must be taken before a.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

The same parts of the graph are walked again for many starting courses, O(V * (V + E)). Kahn's algorithm (take courses with no unmet prerequisites) is O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (topological sort)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the order of courses (LeetCode 210).

The same Kahn loop records the order; if fewer than n courses are taken return an empty list. O(V + E).

FOLLOW-UP 2 The minimum number of semesters if you can take any number of courses per semester.

Run Kahn's algorithm level by level: each level is one semester. The number of levels is the longest chain of prerequisites (return -1 if there is a cycle).

FOLLOW-UP 3 Report an actual cycle.

After Kahn's algorithm, the courses left over all still have an unmet prerequisite among themselves; walk backwards along prerequisites inside that set until a course repeats to extract a cycle.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Course Schedule
#
# You are given numCourses and a list of prerequisites,
# where each element in prerequisites[i] = [a_i, b_i]
# means you must complete course b_i before taking course
# a_i. Write a function canFinish that returns True if
# it's possible to finish all courses, and False if it's
# not. You need to complete all the courses for the result
# to be True. Assume there are no duplicate prerequisites,
# and numCourses is a non-negative number. Examples Input:
# numCourses = 3, prerequisites = [[2, 1], [1, 0]] Output:
# True Explanation: You can complete course 0 first, then
# course 1, and finally course 2. Input: numCourses = 4,
# prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]] Output:
# False Explanation: The prerequisites form a cycle,
# making it impossible to finish all courses.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order of courses (LeetCode 210).
# 2. The minimum number of semesters if you can take any
# number of courses per semester.
# 3. Report an actual cycle.
#
# INTERVIEW NOTES
# Finishing every course is possible exactly when the
# prerequisite graph has no cycle. Kahn's algorithm tests
# that by repeatedly taking courses with no unmet
# prerequisites; if you can take all of them there is no
# cycle, and the order you took them is a valid schedule.
#
# BRUTE FORCE (say it first): DFS from every course
# looking for a path back to itself
# For each course follow dependencies; reaching the start
# again is a cycle. Correct and the oracle; the same parts
# of the graph are walked repeatedly. Time O(V * (V + E)).
# Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): DFS from every course
# looking for a path back to itself | time O(V * (V + E))
def can_finish_brute(n, prereqs):
    adj = defaultdict(list)

    for a, b in prereqs:
        adj[b].append(a)

    def cyc(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(cyc(c) for c in range(n))


# HELPERS for the base question
def build(n, prereqs):
    # (a, b): course b must be taken before course a
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    return adj, indeg


# FOLLOW-UP 1: Return the order of courses (LeetCode 210).
# Answer: The same Kahn loop records the order; if fewer
# than n courses are taken return an empty list. O(V + E).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def find_order(n, prereqs):
    # Kahn's algorithm: take courses with no unmet
    # prerequisite
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    # fewer than n taken: a cycle blocks the rest
    return order if len(order) == n else []


# BASE QUESTION (optimal): Kahn's algorithm (topological
# sort)
# Idea: Count each course's unmet prerequisites and queue
# those with none. Take a course, reduce the counts of the
# courses that need it, and queue any that reach zero.
def can_finish(n, prereqs):
    return len(find_order(n, prereqs)) == n


# FOLLOW-UP 2: The minimum number of semesters if you can
```

```
# take any number of courses per semester.
# Answer: Run Kahn's algorithm level by level: each level
# is one semester. The number of levels is the longest
# chain of prerequisites (return -1 if there is a cycle).
def min_semesters(n, prereqs):
    # any number of courses per semester: one BFS level
    # each
    adj, indeg = build(n, prereqs)
    level = [c for c in range(n) if indeg[c] == 0]
    semesters = taken = 0
    while level:
        semesters += 1
        taken += len(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = nxt
    return semesters if taken == n else -1


# FOLLOW-UP 3: Report an actual cycle.
# Answer: After Kahn's algorithm, the courses left over
# all still have an unmet prerequisite among themselves;
# walk backwards along prerequisites inside that set until
# a course repeats to extract a cycle.
def find_cycle(n, prereqs):
    # the leftover courses all have a leftover
    # prerequisite, so
```

```
    # walking backwards along prerequisites repeats a
    # course
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    while queue:
        c = queue.popleft()
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    left = {c for c in range(n) if indeg[c] > 0}
    if not left:
        return []
    before = defaultdict(list)
    for a, b in prereqs:
        if a in left and b in left:
            before[a].append(b)
    x, walk, pos = min(left), [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    return walk[pos[x]:]


if __name__ == "__main__":
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    diamond = [(1, 0), (2, 0), (3, 1), (3, 2)]
    assert find_order(4, diamond)[0] == 0
    assert find_order(2, [(1, 0), (0, 1)]) == []
    assert can_finish(3, []) and can_finish(0, [])
    assert min_semesters(4, diamond) == 3
    assert min_semesters(2, [(1, 0), (0, 1)]) == -1
    assert min_semesters(3, []) == 1
    pairs = [(1, 0), (2, 1), (1, 2), (3, 2)]
    cycle = find_cycle(4, pairs)
    assert set(cycle) == {1, 2}
    assert find_cycle(2, [(1, 0)]) == []
```

OFFICIAL PAGE TEXT

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples

```
Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]
]
Output: True
Explanation: You can complete course 0 first, then cou
rse 1, and finally course 2.

Input: numCourses = 4, prerequisites = [[3, 2], [2, 1]
, [1, 0], [0, 3]]
Output: False
Explanation: The prerequisites form a cycle, making it
 impossible to finish all courses.

Input: numCourses = 5, prerequisites = [[4, 2], [3, 1]
, [2, 0]]
Output: True
Explanation: You can complete the courses in order fro
m course 0 to course 4.
```

Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.Our solution processes the input data to determine if it's possible to finish all courses given the prerequisite constraints. It constructs a directed graph where nodes represent courses and edges represent the prerequisite relationships. The solution employs a queue to manage nodes with zero in-degrees (courses with no remaining prerequisites) and uses a list to keep track of the number of prerequisites (in-degrees) for each course. For each course with zero in-degrees, the solution dequeues it and processes its neighbors, reducing their in-degrees. If a neighbor's in-degree becomes zero, it is added to the queue. This approach ensures that all courses with no remaining prerequisites are processed and their dependencies are updated accordingly. By maintaining a count of the number of processed courses, the solution can determine if all courses can be finished. If the count of processed courses matches the total number of courses, it implies that there are no cyclic dependencies, and thus all courses can be completed. The solution uses a queue to efficiently manage courses with zero in-degrees and a vector to keep track of in-degrees. This method ensures that each course is processed only once, making the

OFFICIAL PAGE TEXT

approach both clear and efficient. NoteWe are using a BFS (implemented by a queue) for this solution because it helps process nodes level by level (course by course), ensuring that we first handle courses with no dependencies before moving to those that rely on other courses. This approach ensures that we can detect cycles in the course prerequisites, which would make it impossible to complete all courses. Time Complexity: The solution has a time complexity of O(numCourses + numPrerequisites). This is because we process each course (vertex) and each prerequisite pair (edge) exactly once. The time complexity reflects the total number of courses (numCourses) and the number of prerequisite relationships (numPrerequisites). Space Complexity: The solution uses O(numCourses + numPrerequisites) space for the adjacency list and the in-degree vector. The space complexity accounts for the storage needed for both the graph representation and the queue, which in the worst case, contains all courses.No answers contributed by the community yet.

MEDIUM | IN APP LC 105 | Exponent | 29 of 36

Construct a binary tree given its preorder and inorder traversals.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 38.3 30 DAYS 0 PRIORITY 97/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #105)

LeetCode match: LC 105 Construct Binary Tree from Preorder and Inorder Traversal

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/3417/construct-binary-tree-preorder-inorder-traversal

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7]
Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]
Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

APPROACH

Preorder gives the root; hash map of inorder indices splits left/right; recurse with index bounds. O(n).

HOW TO THINK ABOUT IT

Preorder tells you the root (its first element), and inorder tells you how many nodes go left (everything before the root in inorder). That cut gives two independent smaller problems. The only expensive step is locating the root in inorder, which a hash map of value to index makes O(1).

Basis: LeetCode 105, Construct Binary Tree from Preorder and Inorder Traversal (values must be unique).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def build(pre, inord):
    if not pre:
        return None
    root = Node(pre[0])
    i = inord.index(pre[0])  # linear search
    root.left = build(pre[1:i + 1], inord[:i])
    root.right = build(pre[i + 1:],
                       inord[i + 1:])
    return root
```

TIME	SPACE
O(n^2)	O(n^2) for the slices

WHY IT IS SLOW

The linear search and the list copies make a skewed tree O(n^2) in time and space. A dictionary of inorder positions with index bounds is O(n).

OPTIMAL SOLUTION · Hash map of inorder positions and index bounds

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You are given postorder and inorder instead.

The root is the last postorder element, and because postorder reads left, right, root, reading it backwards yields root, right, left, so build the right subtree before the left.

FOLLOW-UP 2 Values can repeat.

Then the tree is not unique and the inorder index of the root is ambiguous. Ask for a rule (or for a preorder with null markers). Preorder plus postorder alone also cannot determine a general binary tree.

FOLLOW-UP 3 The tree is a 100,000-node chain.

The recursion is as deep as the tree. Use an explicit stack: push nodes while walking preorder, and pop back to the parent when the next node's inorder index shows you have finished the left side.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Construct a binary tree given its
# preorder and inorder traversals.
#
# Given two integer arrays, preorder and inorder, which
# represent the pre-order and in-order traversal of a
# binary tree, construct and return the binary tree. Pre-
# order traversal visits nodes in this order: root -> left
# subtree -> right subtree In-order traversal visits nodes
# in this order: left subtree -> root -> right subtree You
# need to construct the binary tree and return its root
# node. Examples preorder = [3, 9, 20, 15, 7], inorder =
# [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7]
# Explanation: Given the two arrays, the binary tree
# constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2],
# inorder = [2, 1] output: [1, 2] Explanation: Given the
# two arrays, the binary tree constructed is: 1 / 2
# Constraints: The number of nodes in the tree is in the
# range [1, 3000]. preorder and inorder consist of unique
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You are given postorder and inorder instead.
# 2. Values can repeat.
# 3. The tree is a 100,000-node chain.
#
# INTERVIEW NOTES
# Preorder tells you the root (its first element), and
# inorder tells you how many nodes go left (everything
# before the root in inorder). That cut gives two
# independent smaller problems. The only expensive step is
# locating the root in inorder, which a hash map of value
# to index makes O(1).
#
# BRUTE FORCE (say it first): Find the root with
# list.index and slice the lists
# Take preorder[0] as the root, find it in inorder with a
# linear search, and recurse on slices. Correct; the
# search and the copying make it quadratic on a skewed
# tree. Time O(n^2). Space O(n^2) for the slices.


# HELPERS for the brute force
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


# BRUTE FORCE (say it first): Find the root with
# list.index and slice the lists | time O(n^2)
def build_tree_slices(preorder, inorder):
    if not preorder:
        return None
    i = inorder.index(preorder[0])
    return TreeNode(
        preorder[0],
        build_tree_slices(
            preorder[1 : 1 + i], inorder[:i]
        ),
        build_tree_slices(
            preorder[1 + i :], inorder[i + 1 :]
        ),
    )


# BASE QUESTION (optimal): Hash map of inorder positions
# and index bounds
# Idea: Map each value to its inorder index.
def build_pre_in(preorder, inorder):
    # preorder: root, left, right. The root's position in
    # the
    # inorder list splits it into the left and right
    # parts.
    where = {v: i for i, v in enumerate(inorder)}
    nxt = 0

    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(preorder[nxt])
        nxt += 1
        mid = where[root.val]
        root.left = build(lo, mid)
        root.right = build(mid + 1, hi)
        return root

    return build(0, len(inorder))


# FOLLOW-UP 1: You are given postorder and inorder
# instead.
# Answer: The root is the last postorder element, and
# because postorder reads left, right, root, reading it
# backwards yields root, right, left, so build the right
# subtree before the left.
def build_post_in(inorder, postorder):
    # postorder: left, right, root. Read it BACKWARDS:
    # root,
    # right, left, so build the right subtree first.
    where = {v: i for i, v in enumerate(inorder)}
    nxt = len(postorder) - 1
```

```
    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(postorder[nxt])
        nxt -= 1
        mid = where[root.val]
        root.right = build(mid + 1, hi)
        root.left = build(lo, mid)
        return root

    return build(0, len(inorder))


# FOLLOW-UP 3: The tree is a 100,000-node chain.
# Answer: The recursion is as deep as the tree. Use an
# explicit stack: push nodes while walking preorder, and
# pop back to the parent when the next node's inorder
# index shows you have finished the left side.
def build_pre_in_iterative(preorder, inorder):
    # a 100,000-node chain: no recursion, an explicit
    # stack
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    stack, in_i = [root], 0
    for val in preorder[1:]:
        node = TreeNode(val)
        if stack[-1].val != inorder[in_i]:
            # the left side of the top node is not
            # finished
            stack[-1].left = node
        else:
```

```
            # pop back to the node whose right child this
            # is
            parent = None
            while stack and stack[-1].val == (
                    inorder[in_i]):
                parent = stack.pop()
                in_i += 1
            parent.right = node
        stack.append(node)
    return root


# HELPERS for the checks below
def inorder_of(root):
    out, stack, node = [], [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.append(node.val)
        node = node.right
    return out


def preorder_of(root):
    out, stack = [], [root] if root else []
    while stack:
        node = stack.pop()
        out.append(node.val)
        for child in (node.right, node.left):
            if child:
                stack.append(child)
    return out


if __name__ == "__main__":
    pre, ino = [3, 9, 20, 15, 7], [9, 3, 15, 20, 7]
```

```
    tree = build_pre_in(pre, ino)
    assert preorder_of(tree) == pre
    assert inorder_of(tree) == ino
    post = [9, 15, 7, 20, 3]
    tree2 = build_post_in(ino, post)
    assert preorder_of(tree2) == pre
    tree3 = build_pre_in_iterative(pre, ino)
```

```
    assert preorder_of(tree3) == pre
    assert inorder_of(tree3) == ino
    assert build_pre_in([], []) is None
    # a very deep chain works without recursion
    n = 50000
    chain = list(range(n))
    deep = build_pre_in_iterative(chain, chain[::-1])
    assert preorder_of(deep) == chain
```

OFFICIAL PAGE TEXT

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples

```
preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20,
 7]
output: [3, 9, 20, null, null, 15, 7]
Explanation:
Given the two arrays, the binary tree constructed is:
        3
       / \
      9   20
         /  \
        15   7
```

```
preorder = [1, 2], inorder = [2, 1]
output: [1, 2]
Explanation:
Given the two arrays, the binary tree constructed is:
    1
   /
  2
```

OFFICIAL PAGE TEXT

Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order. Our solution to construct a binary tree from pre-order and in-order traversal arrays uses a recursive approach. We first identify the root node from the pre-order array, then split the in-order array into left and right subtrees based on the root's position in the in-order array. We continue this process recursively to build the entire binary tree. We start by picking the first element of the preorder array as the root of the tree. The index of this root element in the inorder array helps us divide the tree into left and right subtrees. The elements to the left of the root in the inorder array form the left subtree, and the elements to the right form the right subtree. We then recursively build the left and right subtrees using the next available elements in the preorder array. To ensure efficient lookups of the root's index in the inorder array, we create a hash map (inorder_index_map) that maps values to their indices in the inorder array. This allows us to retrieve the position of the root in constant time. NoteRecursion: Base Case: If there are no elements to construct the tree (i.e., the preorder or inorder array is empty), we return nullptr. Recursive Step: We extract the root from the preorder

array and find its position in the inorder array using the hash map. This splits the inorder array into left and right subtrees. We then recursively build the left and right subtrees and attach them to the root. By maintaining the correct indices for both the pre-order and in-order arrays, we can recursively construct the entire tree. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the tree. This is because we traverse both the preorder and inorder arrays once, and we use a hash map for constant-time lookups of root indices in the inorder array. Space Complexity: The solution uses O(n) space for the hash map that stores the indices of elements in the inorder array, and the recursion stack requires O(h) space, where h is the height of the tree. In the worst case (a skewed tree), the height of the tree can be O(n), so the total space complexity is O(n).No answers contributed by the community yet.

MEDIUM | IN APP LC 53 | Exponent | 30 of 36

★ TOP 75: rank 20 in the Medium and Hard priority list

★ Find the maximum subarray sum.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 64.4 30 DAYS 41.1 PRIORITY 14/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #53)

LeetCode match: LC 53 Maximum Subarray

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1522 interviews

Original: https://www.aced.io/questions/973/maximum-subarray-sum

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

APPROACH

Kadane's: cur = max(x, cur + x); best = max(best, cur). O(n).

HOW TO THINK ABOUT IT

The best subarray ending at position i either extends the best one ending at i - 1 or starts fresh at i; it extends only if the previous best is positive. So keep one number, the best sum ending here, and one answer: cur = max(x, cur + x), best = max(best, cur).

Basis: LeetCode 53, Maximum Subarray (Kadane's algorithm). The subarray must be non-empty.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    best = -INF
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2) subarrays. Kadane's algorithm keeps the best sum ending at each position (extend or restart) in O(n) time and O(1) space.

OPTIMAL SOLUTION · Kadane's algorithm

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subarray itself.

Remember where the current run started: when starting fresh at i set start = i, and when cur beats best record (start, i). Same single pass.

FOLLOW-UP 2 The array is circular (LeetCode 918).

The best circular subarray is either an ordinary one or the whole array minus the worst ordinary one (the minimum subarray): max(kadane max, total - kadane min), taking care of the all-negative case.

FOLLOW-UP 3 Divide and conquer instead of Kadane.

Split the array; the best subarray is entirely left, entirely right, or crosses the middle (best suffix of the left plus best prefix of the right). T(n) = 2T(n/2) + O(n) = O(n log n); useful when the data is split across machines.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the maximum subarray sum.
#
# Given an array of integers nums, write a function
# maxSubarraySum to find the maximum sum of a contiguous
# subarray within the array and return that maximum sum.
# The subarray must be contiguous, meaning that the
# elements must appear consecutively in the original
# array. Examples Input: nums = [2, 3, -2, 4] Output: 7
# Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7. Input:
# nums = [1, -1, -5, -4] Output: 1 Explanation: The
# maximum sum is 1, which is the single element with the
# highest value.Challenge 1Can you come up with a solution
# with a O(n) time complexity?Challenge 2Can you come up
# with a solution with a O(1) space complexity?
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray itself.
# 2. The array is circular (LeetCode 918).
# 3. Divide and conquer instead of Kadane.
#
```

```
# INTERVIEW NOTES
# The best subarray ending at position i either extends
# the best one ending at i - 1 or starts fresh at i; it
# extends only if the previous best is positive. So keep
# one number, the best sum ending here, and one answer:
# cur = max(x, cur + x), best = max(best, cur).
#
# BRUTE FORCE (say it first): Try every subarray
# For each start extend the end with a running sum and
# keep the maximum. Correct and the oracle; quadratic.
# Time O(n^2). Space O(1).


# BRUTE FORCE (say it first): Try every subarray | time
# O(n^2)
def max_subarray_brute(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

```
# BASE QUESTION (optimal): Kadane's algorithm
# Idea: cur is the best sum of a subarray ending at the
# current element: either the element alone or the element
# added to the previous cur. best is the maximum cur seen.
def max_subarray(nums):
    # Kadane: the best sum of a run ending here either
    # extends
    # the previous run or starts again at this element
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best


# FOLLOW-UP 1: Return the subarray itself.
# Answer: Remember where the current run started: when
# starting fresh at i set start = i, and when cur beats
# best record (start, i). Same single pass.
def max_subarray_range(nums):
    # also say where it is: remember where the run started
    cur, start = nums[0], 0
    best, span = nums[0], (0, 0)
    for i in range(1, len(nums)):
        if nums[i] > cur + nums[i]:
            cur, start = nums[i], i
        else:
            cur += nums[i]
        if cur > best:
            best, span = cur, (start, i)
    return best, span


# FOLLOW-UP 2: The array is circular (LeetCode 918).
# Answer: The best circular subarray is either an ordinary
# one or the whole array minus the worst ordinary one (the
# minimum subarray): max(kadane max, total - kadane min),
# taking care of the all-negative case.
def max_circular(nums):
    # circular array (LeetCode 918): the best run either
    # does not
    # wrap, or wraps and equals total minus the WORST run
    best = max_subarray(nums)
    total = sum(nums)
    worst = -max_subarray([-x for x in nums])
```

```
    # all negative: the wrap would be empty, so use best
    if total == worst:
```

```
        return best
    return max(best, total - worst)


# FOLLOW-UP 3: Divide and conquer instead of Kadane.
# Answer: Split the array; the best subarray is entirely
# left, entirely right, or crosses the middle (best suffix
# of the left plus best prefix of the right). T(n) =
# 2T(n/2) + O(n) = O(n log n); useful when the data is
# split across machines.
def max_subarray_dc(nums, lo=0, hi=None):
    # divide and conquer: left, right, or crossing the
    # middle
    hi = len(nums) if hi is None else hi
    if hi - lo == 1:
        return nums[lo]
    mid = (lo + hi) // 2
    left_best = right_best = float("-inf")
    run = 0
    for i in range(mid - 1, lo - 1, -1):
        run += nums[i]
        left_best = max(left_best, run)
    run = 0
    for i in range(mid, hi):
        run += nums[i]
        right_best = max(right_best, run)
    return max(max_subarray_dc(nums, lo, mid),
               max_subarray_dc(nums, mid, hi),
               left_best + right_best)


if __name__ == "__main__":
    assert max_subarray([2, 3, -2, 4]) == 7
    assert max_subarray([-3, -1, -2]) == -1
    assert max_subarray([5]) == 5
    mixed = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
    assert max_subarray_range(mixed) == (6, (3, 6))
    assert max_circular([5, -3, 5]) == 10
    assert max_circular([-3, -2, -3]) == -2
    assert max_circular([1, -2, 3, -2]) == 3
    samples = ([2, 3, -2, 4], [-1, -2], [1, -1, 1, -1, 5])
    for arr in samples:
        assert max_subarray_dc(arr) == max_subarray(arr)
```

OFFICIAL PAGE TEXT

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples

```
Input: nums = [2, 3, -2, 4]
Output: 7
Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.

Input: nums = [1, -1, -5, -4]
Output: 1
Explanation: The maximum sum is 1, which is the single
 element with the highest value.
```

OFFICIAL PAGE TEXT

Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?Our solution for finding the maximum subarray sum processes the input array by iterating through it while maintaining two variables: max_current and max_global. The max_current variable keeps track of the maximum sum of the subarray ending at the current position, while max_global stores the maximum sum encountered so far. For each element in the array, we update max_current by taking the maximum of the current element alone or the sum of the current element and max_current. This decision ensures that max_current always holds the maximum sum of the subarray that ends at the current element. We then update max_global to be the maximum of max_global and max_current. If the input array is empty, the function returns 0 since there are no subarrays to consider. This approach uses a simple iteration and a couple of variables, making it both straightforward and efficient. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (comparison and addition) for each element. Space Complexity: The solution uses O(1) space since it only requires a fixed amount of extra space

for the max_current and max_global variables, regardless of the input size. This makes the solution very efficient in terms of space usage. NoteThis algorithm is known as Kadane's Algorithm.No answers contributed by the community yet.

MEDIUM | IN APP LC 416 | Exponent | 31 of 36

Partition an array into two sub-arrays with equal sum.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 46.6 30 DAYS 0 PRIORITY 75/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #416)

LeetCode match: LC 416 Partition Equal Subset Sum

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/969/partition-array-equal-sum

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples
Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

APPROACH

Subset-sum DP to total/2 (boolean array or bitset); odd totals are impossible. O(n * sum/2).

HOW TO THINK ABOUT IT

Two equal parts mean each sums to total / 2, so if the total is odd the answer is no. Otherwise the question is whether some subset reaches total / 2, which is subset-sum. A bitset where bit s says 'sum s is reachable' updates with one shift and one OR per number.

Basis: LeetCode 416, Partition Equal Subset Sum (the two parts are subsets, not contiguous sub-arrays; the statement says sub-arrays, so the contiguous reading is given as a follow-up).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    total = sum(nums)
    if total % 2:
        return False
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            if sum(subset) == total // 2:
                return True
    return False
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n subsets. Subset-sum as a reachability table (or a bitset shifted by each number) is O(n * sum).

OPTIMAL SOLUTION · Subset-sum as a bitset

TIME	SPACE
O(n * sum / 64) with big-integer shifts (O(n * sum) as a boolean table)	O(sum)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the two subsets.

Keep, for each reachable sum, the list of indices (or a parent pointer) that first reached it; read off the subset for total / 2 and give the rest to the other side.

FOLLOW-UP 2 Minimise the difference between the two parts (LeetCode 1049).

Compute all reachable subset sums with the same bitset and choose the reachable s closest to total / 2: the answer is total - 2s.

FOLLOW-UP 3 The parts must be contiguous sub-arrays (a split point).

Then only one split can work: compute prefix sums and check whether some prefix equals total / 2. O(n), O(1) space; no DP is needed.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Partition an array into two sub-arrays
# with equal sum.
#
# Given an array of integers, determine if it can be
# partitioned into two subsets such that the sum of the
# elements in each subset is equal. A partition is
# considered valid if it divides the array into two
# subsets where both subsets have the same sum. Examples
# Input: [2, 4, 11, 5] Output: True Explanation: The array
# can be partitioned into two subsets [2, 4, 5] and [11]
# with equal sum. Input: [2, 7, 3, 1] Output: False
# Explanation: The array cannot be partitioned into two
# subsets with equal sums.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two subsets.
# 2. Minimise the difference between the two parts
# (LeetCode 1049).
# 3. The parts must be contiguous sub-arrays (a split
# point).
#
# INTERVIEW NOTES
# Two equal parts mean each sums to total / 2, so if the
# total is odd the answer is no. Otherwise the question is
# whether some subset reaches total / 2, which is subset-
# sum. A bitset where bit s says 'sum s is reachable'
# updates with one shift and one OR per number.
#
# BRUTE FORCE (say it first): Try every subset
# Enumerate all subsets and check whether any sums to half
# the total. Exact and the oracle; exponential. Time O(2^n
# * n). Space O(n).


import itertools


# BRUTE FORCE (say it first): Try every subset | time
# O(2^n * n)
def can_partition_brute(nums):
    total = sum(nums)
    if total % 2:
        return False
```

```
    return any(
        sum(c) == total // 2
        for r in range(len(nums) + 1)
        for c in itertools.combinations(nums, r)
    )


# BASE QUESTION (optimal): Subset-sum as a bitset
# Idea: reach is an integer whose bit s is set when some
# subset sums to s; start with bit 0. For each number x do
# reach |= reach << x. The answer is whether bit total / 2
# is set.
def can_partition(nums):
    # reachable subset sums as the bits of one integer
    total = sum(nums)
    if total % 2:
        return False
    reach = 1
    for x in nums:
        # every old sum s also gives s + x
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)


# FOLLOW-UP 1: Return the two subsets.
# Answer: Keep, for each reachable sum, the list of
# indices (or a parent pointer) that first reached it;
# read off the subset for total / 2 and give the rest to
# the other side.
def partition_subsets(nums):
    # also return the two subsets: remember which item
    # first
    # reached each sum
    total = sum(nums)
    if total % 2:
        return None
    half = total // 2
    first = {0: -1}
    for i, x in enumerate(nums):
        for s in list(first):
            if s + x <= half and s + x not in first:
                first[s + x] = i
    if half not in first:
        return None
    chosen, s = [], half
    while s:
        i = first[s]
        chosen.append(i)
        s -= nums[i]
    left = sorted(chosen)
    rest = [i for i in range(len(nums))
            if i not in chosen]
    return ([nums[i] for i in left],
            [nums[i] for i in rest])


# FOLLOW-UP 2: Minimise the difference between the two
# parts (LeetCode 1049).
# Answer: Compute all reachable subset sums with the same
# bitset and choose the reachable s closest to total / 2:
# the answer is total - 2s.
def min_difference(nums):
    # LeetCode 1049: make the two sums as close as
    # possible
    total = sum(nums)
    reach = 1
    for x in nums:
        reach |= reach << x
    sums = range(total // 2 + 1)
    best = max(s for s in sums if reach >> s & 1)
    return total - 2 * best


# FOLLOW-UP 3: The parts must be contiguous sub-arrays (a
# split point).
# Answer: Then only one split can work: compute prefix
# sums and check whether some prefix equals total / 2.
# O(n), O(1) space; no DP is needed.
def can_split_contiguous(nums):
    # the parts must be a prefix and a suffix: no DP
    # needed
    total, prefix = sum(nums), 0
    if total % 2:
        return False
    for x in nums:
        prefix += x
        if prefix * 2 == total:
            return True
    return False


if __name__ == "__main__":
    # 2 + 4 + 5 = 11 = the other part
    assert can_partition([2, 4, 11, 5]) is True
    assert can_partition([1, 2, 5]) is False
    assert can_partition([]) is True
    assert can_partition([1]) is False
    left, right = partition_subsets([1, 5, 11, 5])
    assert sum(left) == sum(right) == 11
    assert sorted(left + right) == [1, 5, 5, 11]
    assert partition_subsets([1, 2, 5]) is None
    assert min_difference([1, 6, 11, 5]) == 1
    assert min_difference([2, 4, 11, 5]) == 0
    assert can_split_contiguous([1, 2, 3, 6])
    assert not can_split_contiguous([1, 5, 11, 5])
```

OFFICIAL PAGE TEXT

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples

```
Input: [2, 4, 11, 5]
Output: True
Explanation: The array can be partitioned into two sub
sets [2, 4, 5] and [11] with equal sum.

Input: [2, 7, 3, 1]
Output: False
Explanation: The array cannot be partitioned into two
subsets with equal sums.
```

OFFICIAL PAGE TEXT

Our solution processes the input array by first calculating the total sum of the array elements and then determining if it can be partitioned into two subsets with equal sums. The core idea is based on dynamic programming. Calculate Total Sum: We start by computing the total sum of the array elements. If the total sum is odd, it is immediately impossible to partition it into two subsets with equal sums, so we return false. Define Target Sum: If the total sum is even, we calculate the target sum for each subset, which is half of the total sum. Dynamic Programming Array: We use a boolean array dp where dp[i] indicates whether a subset sum of i is achievable. We initialize dp[0] to true because a subset sum of 0 is always achievable (by taking no elements). Iterate Through the Array: For each number in the array, we update the dp array in reverse order (from target down to the current number). This reverse iteration ensures that each number is only used once in the current update. If a subset sum i - num was achievable, then adding the current number num makes dp[i] achievable. We update dp[i] to true accordingly. Check Final Result: After processing all numbers, if dp[target] is true, it means there is a subset with the required sum equal to target, indicating that the array can be partitioned into two subsets with equal sums. Otherwise, we return false. The use of a boolean array ensures that we efficiently keep

track of achievable subset sums. Time Complexity: The solution has a time complexity of O(n * target), where n is the number of elements in the array and target is half of the total sum. This is because we iterate through the array and update the dp array for each element. Space Complexity: The solution uses O(target) space for the dp array, where target is half of the total sum. This space is needed to store the achievable subset sums.No answers contributed by the community yet.

MEDIUM | IN APP LC 46 | Exponent | 32 of 36

Permutations

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 49.8 30 DAYS 0 PRIORITY 70/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #46)

LeetCode match: LC 46 Permutations

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +25 interviews

Original: https://www.aced.io/questions/2212/leetcode-medium-permutations-dp

QUESTION

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example Input: nums = [1, 2, 3] Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

APPROACH

Backtracking with a used[] array (or swap-based in place). n! outputs.

HOW TO THINK ABOUT IT

The output has n! items, so no algorithm can beat O(n * n!); the question is only whether you generate them without waste. Build a permutation one position at a time, marking which elements are used, and undo the choice when you come back (backtracking). Every follow-up (duplicates, k-th, next) is a variation on 'what are the legal choices at this position'.

Basis: LeetCode 46 (Permutations), 47 (with duplicates), 31 (Next Permutation) and 60 (Permutation Sequence).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    result = []
    for seq in product(nums, repeat=len(nums)):
        if len(set(seq)) == len(nums):
            result.append(list(seq))
    return result
```

TIME	SPACE
O(n^n * n)	O(n!)

WHY IT IS SLOW

n^n sequences when only n! are valid, so almost all the work is wasted. Backtracking with a used marker only builds valid prefixes, O(n * n!).

OPTIMAL SOLUTION · Backtracking with a used array

TIME	SPACE
O(n * n!)	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input has duplicates (Permutations II).

Sort first, and skip a value that equals the previous one unless the previous one is already used in the current prefix. That forces equal values to appear in a fixed order and removes duplicates without a set.

FOLLOW-UP 2 Return the next lexicographic permutation in place.

Find the last position i with a[i] < a[i + 1], swap a[i] with the smallest larger value to its right, then reverse the suffix. O(n), O(1) space. Repeat to enumerate in order.

FOLLOW-UP 3 Return only the k-th permutation of 1..n.

Use the factorial number system: the first digit is index (k - 1) // (n - 1)! of the remaining digits, then continue with the remainder. O(n^2), no enumeration.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Permutations
#
# Given an array of distinct integers, return all possible
# permutations of the array. A permutation of an array is
# a rearrangement of its elements. Example Input: nums =
# [1, 2, 3] Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2,
# 3, 1], [3, 1, 2], [3, 2, 1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input has duplicates (Permutations II).
# 2. Return the next lexicographic permutation in place.
# 3. Return only the k-th permutation of 1..n.
#
# INTERVIEW NOTES
# The output has n! items, so no algorithm can beat O(n *
# n!); the question is only whether you generate them
# without waste. Build a permutation one position at a
# time, marking which elements are used, and undo the
# choice when you come back (backtracking). Every follow-
# up (duplicates, k-th, next) is a variation on 'what are
# the legal choices at this position'.
#
# BRUTE FORCE (say it first): Generate all n^n sequences
# and keep the valid ones
# Enumerate every sequence of length n over the values and
# filter those with all values distinct. It shows the idea
# but wastes almost all of its work (n^n versus n!). Time
# O(n^n * n). Space O(n!).


import itertools
from math import factorial


# BRUTE FORCE (say it first): Generate all n^n sequences
# and keep the valid ones | time O(n^n * n)
def permute_brute(nums):
    return [
        list(seq)
        for seq in itertools.product(
            nums, repeat=len(nums)
        )
        if len(set(seq)) == len(nums)
    ]
```

```
# BASE QUESTION (optimal): Backtracking with a used array
# Idea: Fill positions left to right. At each position try
# every unused value, mark it, recurse, and unmark. Only
# valid prefixes are ever extended, so every step leads to
# a permutation.
def permute(nums):
    # backtracking: choose an unused element, recurse,
    # undo
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True
                cur.append(x)
                go()
                cur.pop()
                used[i] = False

    go()
    return out


# FOLLOW-UP 1: The input has duplicates (Permutations II).
# Answer: Sort first, and skip a value that equals the
# previous one unless the previous one is already used in
# the current prefix. That forces equal values to appear
# in a fixed order and removes duplicates without a set.
def permute_unique(nums):
    # duplicates: sort, and skip a value equal to the
    # previous
    # one unless that one is already in the current prefix
    nums = sorted(nums)
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if used[i]:
                continue
            same = i and nums[i] == nums[i - 1]
            if same and not used[i - 1]:
                continue
            used[i] = True
            cur.append(x)
            go()
            cur.pop()
            used[i] = False

    go()
    return out


# FOLLOW-UP 2: Return the next lexicographic permutation
# in place.
# Answer: Find the last position i with a[i] < a[i + 1],
# swap a[i] with the smallest larger value to its right,
# then reverse the suffix. O(n), O(1) space. Repeat to
# enumerate in order.
def next_permutation(a):
    # in place: O(n) time, O(1) space; False when it was
    # the last
    i = len(a) - 2
    # the last position that can still increase
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1
    if i < 0:
        a.reverse()
        return False
    j = len(a) - 1
    while a[j] <= a[i]:
```

```
        j -= 1
    a[i], a[j] = a[j], a[i]
    # the suffix is decreasing: make it the smallest
    a[i + 1:] = reversed(a[i + 1:])
    return True


# FOLLOW-UP 3: Return only the k-th permutation of 1..n.
# Answer: Use the factorial number system: the first digit
# is index (k - 1) // (n - 1)! of the remaining digits,
# then continue with the remainder. O(n^2), no
# enumeration.
def kth_permutation(n, k):
```

```
    # the k-th (1-based) permutation of 1..n without
    # listing
    digits, out, k = list(range(1, n + 1)), [], k - 1
    for i in range(n, 0, -1):
        block = factorial(i - 1)
        out.append(digits.pop(k // block))
        k %= block
    return out


if __name__ == "__main__":
    assert permute([1, 2, 3]) == [
        [1, 2, 3], [1, 3, 2], [2, 1, 3],
        [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    assert permute([]) == [[]]
    assert permute_unique([1, 1, 2]) == [
        [1, 1, 2], [1, 2, 1], [2, 1, 1]]
    a = [1, 2, 3]
    assert next_permutation(a) and a == [1, 3, 2]
    last = [3, 2, 1]
    assert not next_permutation(last)
    assert last == [1, 2, 3]
    # walking next_permutation lists every order
    seen, p = [], [1, 2, 3]
    while True:
        seen.append(p[:])
        if not next_permutation(p):
            break
    assert seen == permute([1, 2, 3])
    assert kth_permutation(3, 3) == [2, 1, 3]
    assert kth_permutation(4, 9) == [2, 3, 1, 4]
```

OFFICIAL PAGE TEXT

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example

```
Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [
3, 1, 2], [3, 2, 1]]
```

OFFICIAL PAGE TEXT

Our solution generates all permutations of a given array by utilizing a recursive backtracking approach. The idea is to explore every possible way of arranging the elements by swapping them at each recursive call. Here's a breakdown of how the algorithm works: We define a helper function (backtrack) that recursively builds the permutations. This function takes two parameters: start (the index from where we start permuting) and end (the length of the array). The base case occurs when start == end, meaning that a complete permutation has been formed. We add a copy of the current permutation (stored in nums) to the result. In each recursive call, we iterate over the elements starting from the start index. For each element at position i, we: Swap the elements at start and i to bring a new element into the current position. Recursively call backtrack with start + 1, moving to the next index to fill. After the recursive call, we swap back the elements to restore the original order and continue exploring other possibilities. This process continues until all permutations are generated. The swapping ensures that we explore every possible combination of the elements. Time Complexity: The time complexity of this solution is O(n * n!), where n is the number of elements in the array. The reason for this is that generating each permutation takes O(n) time, and there are n! permutations. Space Complexity: The space

complexity is O(n!) due to the storage of all permutations in the result list. Additionally, the recursion stack uses O(n) space for the recursive calls, but this is dominated by the result storage.No answers contributed by the community yet.

MEDIUM | IN APP LC 56 | Exponent | 33 of 36

★ TOP 75: rank 11 in the Medium and Hard priority list

★ Merge Intervals

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 11/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #56)

LeetCode match: LC 56 Merge Intervals

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +1219 interviews

Original: https://www.aced.io/questions/1602/merge-intervals

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each

interval, compare it with the last interval in th

APPROACH

Sort by start and merge overlapping intervals in one pass. O(n log n).

HOW TO THINK ABOUT IT

After sorting by start, an interval can overlap only the interval just before it in the output, because everything earlier ends before that one starts. So one pass with a 'current merged interval' is enough: extend its end, or start a new one.

Basis: LeetCode 56, Merge Intervals (intervals sharing a point merge).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

BRUTE FORCE

TIME	SPACE
O(n^2) or worse	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Sort by start and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Insert one interval into an already merged list (LeetCode 57).

Copy the intervals that end before it, merge the ones that overlap, copy the rest: O(n) with no sort.

FOLLOW-UP 2 Intervals of touching integers ([1, 2] and [3, 4]) should merge.

Change the overlap test to start <= last_end + 1. For real-valued ranges that would be wrong, so state which you assume.

FOLLOW-UP 3 Too many intervals for memory.

Sort externally (external merge sort) and stream the merge: the one-pass sweep needs only the current interval in memory. **Cost:** O(n log n) external sort with bounded memory runs, then an O(n) streaming merge with O(1) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Merge Intervals
#
# Merge Intervals. An interval is a representation of a
# range of values along a number line. For example, [1,3]
# represents all numbers between 1 and 3, inclusive. Given
# a collection of intervals, the task is to merge all
# overlapping intervals. For example, consider the
# intervals [1,3],[2,6],[8,10],[15,18]. The function
# should return merged intervals where overlapping
# intervals are combined. In this case, intervals [1,3]
# and [2,6] overlap, so they should be merged into [1,6].
# Examples intervals = [[1,3], [2,6], [8,10], [15,18]]
# output: [[1,6], [8,10], [15,18]] intervals = [[1,4],
# [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]]
# output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]]
# output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals
# function takes an array of intervals and merges all
# overlapping intervals into a single interval.
#
# FOLLOW-UPS (answered below, in this order):
```

```python
# 1. Insert one interval into an already merged list
# (LeetCode 57).
# 2. Intervals of touching integers ([1, 2] and [3, 4])
# should merge.
# 3. Too many intervals for memory.
#
# INTERVIEW NOTES
# After sorting by start, an interval can overlap only the
# interval just before it in the output, because
# everything earlier ends before that one starts. So one
# pass with a 'current merged interval' is enough: extend
# its end, or start a new one.
#
# BRUTE FORCE (say it first): Repeatedly merge any
# overlapping pair
# Look for two intervals that overlap, replace them with
# their union, and repeat until none overlap. Correct and
# the oracle; many passes over all pairs. Time O(n^2) or
# worse. Space O(n).


import heapq


# BRUTE FORCE (say it first): Repeatedly merge any
```

```python
# overlapping pair | time O(n^2) or worse
def merge_pairwise(intervals):
    items = [list(i) for i in intervals]
    changed = True
    while changed:
        changed = False
        for i in range(len(items)):
            for j in range(i + 1, len(items)):
                if (
                    items[i][0] <= items[j][1]
                    and items[j][0] <= items[i][1]
                ):
                    items[i] = [
                        min(items[i][0], items[j][0]),
                        max(items[i][1], items[j][1]),
                    ]
                    del items[j]
                    changed = True
                    break
            if changed:
                break
    return sorted(items)


# BASE QUESTION (optimal): Sort by start and sweep
# Idea: Sort by start. If an interval starts at or before
# the end of the last output interval, extend that end
# with max; otherwise append it. The sort guarantees
# nothing earlier can overlap the new interval.
def merge_intervals(intervals, touching_ints=False):
    out = []
    # sort by start, then extend the last interval while
    # they overlap
    for s, e in sorted(intervals):
        gap = 1 if touching_ints else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out


# FOLLOW-UP 1: Insert one interval into an already merged
# list (LeetCode 57).
# Answer: Copy the intervals that end before it, merge the
```

```python
# ones that overlap, copy the rest: O(n) with no sort.
def insert_interval(merged, new):
    # LeetCode 57: already merged and sorted, so O(n) and
    # no sort
    out, i, n = [], 0, len(merged)
    # 1) everything that ends before the new one starts
    while i < n and merged[i][1] < new[0]:
        out.append(merged[i])
        i += 1
    # 2) everything that overlaps it is absorbed
    s, e = new
    while i < n and merged[i][0] <= e:
        s, e = min(s, merged[i][0]), max(e, merged[i][1])
        i += 1
    out.append([s, e])
    # 3) the rest
    return out + merged[i:]


# FOLLOW-UP 3: Too many intervals for memory.
# Answer: Sort externally (external merge sort) and stream
# the merge: the one-pass sweep needs only the current
# interval in memory. Cost: O(n log n) external sort with
# bounded memory runs, then an O(n) streaming merge with
# O(1) memory.
def merge_stream(sorted_intervals):
    # intervals arrive sorted by start (for example after
    # an
    # external sort): only the current one is kept in
    # memory
    cur = None
    for s, e in sorted_intervals:
        if cur and s <= cur[1]:
            cur[1] = max(cur[1], e)
        else:
            if cur:
                yield tuple(cur)
            cur = [s, e]
    if cur:
        yield tuple(cur)


# FOLLOW-UP 3: Too many intervals for memory.
# Answer: Sort externally (external merge sort) and stream
# the merge: the one-pass sweep needs only the current
```

```python
# interval in memory. Cost: O(n log n) external sort with
# bounded memory runs, then an O(n) streaming merge with
# O(1) memory.
def merge_many_sorted(sources):
    # several already-sorted files: a heap merge, then the
    # sweep
    return list(merge_stream(heapq.merge(*sources)))


if __name__ == "__main__":
    spans = [[1, 3], [2, 6], [8, 10], [15, 18]]
    got = merge_intervals(spans)
    assert got == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    got = merge_intervals([[1, 2], [3, 4]])
    assert got == [[1, 2], [3, 4]]
    got = merge_intervals([[1, 2], [3, 4]], True)
    assert got == [[1, 4]]
    got = insert_interval([[1, 3], [6, 9]], [2, 5])
    assert got == [[1, 5], [6, 9]]
    assert insert_interval([], [5, 7]) == [[5, 7]]
    stream = list(merge_stream([(1, 3), (2, 6), (8, 10)]))
    assert stream == [(1, 6), (8, 10)]
    parts = [[(1, 3), (8, 9)], [(2, 4), (9, 12)]]
    assert merge_many_sorted(parts) == [(1, 4), (8, 12)]
```

OFFICIAL PAGE TEXT

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
output: [[1,6], [8,10], [15,18]]

intervals = [[1,4], [4,5]]
output: [[1,5]]

intervals = [[1,4], [2,3]]
output: [[1,4]]

intervals = [[1,2], [3,4], [5,6], [7,8]]
output: [[1,2], [3,4], [5,6], [7,8]]
```

The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in the result list. If the current interval overlaps with the last interval in the result (i.e., the start of the current interval is less than or equal to the end of the last interval), merge them by updating the end of the last interval in the result to the maximum end time of both intervals. If there is no overlap, add the current interval to the result list as a separate interval. Return the Result: After processing all intervals, the result list contains the merged intervals. Time Complexity: The dominant factor in the time complexity is the sorting step, which takes O(n log n), where n is the number of intervals. The merging process involves a single pass through the sorted intervals, which takes O(n) time. Therefore, the overall time complexity is O(n log n). Space Complexity: The space complexity is O(n), which accounts for the space required to store the result list. This is because, in the worst case, none of the intervals overlap, and the result list will contain all the original intervals.In this mock interview video, Lewin (Dropbox SWE), answers the interview question, "Given an array of intervals, merge the overlapping intervals and return an array of the non-overlapping intervals that cover all intervals in the input."No answers contributed by the community yet.

MEDIUM | IN APP LC 49 | Exponent | 34 of 36

★ TOP 75: rank 23 in the Medium and Hard priority list

★ Group anagrams

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.7 30 DAYS 51.4 PRIORITY 15/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #49)

LeetCode match: LC 49 Group Anagrams (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4676/group-anagrams

QUESTION

Group anagrams

APPROACH

Hash map keyed by sorted letters (or a 26-count tuple) to lists of words. O(n * k log k).

HOW TO THINK ABOUT IT

Anagrams are strings with the same letters in any order, so they have the same canonical form: the sorted letters, or the count of each letter. Use that as a dictionary key and group in one pass; you never compare two strings directly.

Basis: LeetCode 49, Group Anagrams.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    groups = []
    for word in words:
        key = sorted(word)
        for group in groups:
            if sorted(group[0]) == key:
                group.append(word)
                break
        else:
            groups.append([word])
    return groups
```

TIME	SPACE
O(n * g * L log L)	O(n)

WHY IT IS SLOW

Each word is compared with many groups, O(n * groups * L log L). Using the sorted letters (or a letter-count tuple) as a dictionary key groups all words in one pass.

OPTIMAL SOLUTION · Dictionary keyed by the sorted letters

TIME	SPACE
O(n * L log L)	O(n * L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the key O(L) instead of O(L log L).

Use a tuple of 26 letter counts as the key (no sort). Total O(n * L); for large alphabets use a Counter turned into a frozenset of items or a sorted tuple of (char, count).

FOLLOW-UP 2 Case-insensitive, Unicode, or ignore spaces and punctuation.

Normalise first (casefold, strip non-letters, Unicode normalisation) and then compute the key. Say what counts as a letter.

FOLLOW-UP 3 The words are in many files.

Compute the key on each machine and hash-partition by key so every anagram group lands on one worker, which then groups locally (a MapReduce shuffle).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Group anagrams
#
# Group anagrams
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the key O(L) instead of O(L log L).
# 2. Case-insensitive, Unicode, or ignore spaces and
# punctuation.
# 3. The words are in many files.
#
# INTERVIEW NOTES
# Anagrams are strings with the same letters in any order,
# so they have the same canonical form: the sorted
# letters, or the count of each letter. Use that as a
# dictionary key and group in one pass; you never compare
# two strings directly.
#
# BRUTE FORCE (say it first): Compare each word with a
# representative of every group
# For each word compare its sorted letters with those of
# the first word of every existing group; start a new
# group when none match. Correct and the oracle; each word
# is compared with many groups. Time O(n * g * L log L).
# Space O(n).


import unicodedata
from collections import defaultdict


# BRUTE FORCE (say it first): Compare each word with a
# representative of every group | time O(n * g * L log L)
def group_anagrams_pairwise(words):
    groups = []
    for w in words:
        for g in groups:
            if sorted(g[0]) == sorted(w):
                g.append(w)
                break
        else:
            groups.append([w])
    return groups


# BASE QUESTION (optimal): Dictionary keyed by the sorted
# letters
# Idea: Sort the letters of each word and use the
# resulting string as the key; words with the same key are
# anagrams. One pass, one dictionary.
def group_by_sorting(words):
    # anagrams share the same letters in sorted order
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())


# FOLLOW-UP 1: Make the key O(L) instead of O(L log L).
# Answer: Use a tuple of 26 letter counts as the key (no
# sort). Total O(n * L); for large alphabets use a Counter
# turned into a frozenset of items or a sorted tuple of
# (char, count).
def group_by_counts(words):
    # key = 26 letter counts: O(L) per word instead of O(L
    # log L)
    groups = defaultdict(list)
    for w in words:
        counts = [0] * 26
        for ch in w:
            counts[ord(ch) - ord("a")] += 1
        groups[tuple(counts)].append(w)
    return list(groups.values())


# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces
# and punctuation.
# Answer: Normalise first (casefold, strip non-letters,
# Unicode normalisation) and then compute the key. Say
# what counts as a letter.
def normalise(word):
    # case, accents, spaces and punctuation: decide what
    # counts
    text = unicodedata.normalize("NFKD", word).casefold()
    return "".join(ch for ch in text if ch.isalpha())


# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces
# and punctuation.
# Answer: Normalise first (casefold, strip non-letters,
# Unicode normalisation) and then compute the key. Say
# what counts as a letter.
def group_normalised(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(normalise(w)))].append(w)
    return list(groups.values())


# FOLLOW-UP 3: The words are in many files.
# Answer: Compute the key on each machine and hash-
# partition by key so every anagram group lands on one
# worker, which then groups locally (a MapReduce shuffle).
def shard_of(word, workers):
    # many files / machines: hash the key so one anagram
    # group
```

```
    # always lands on the same worker
    key = "".join(sorted(word))
    return sum(map(ord, key)) % workers


if __name__ == "__main__":
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    want = [["eat", "tea", "ate"], ["tan", "nat"],
            ["bat"]]
    assert group_by_sorting(words) == want
    assert group_by_counts(words) == want
    assert group_by_sorting([]) == []
    assert group_by_sorting([""]) == [[""]]
    mixed = ["Listen", "Silent", "enlist", "a gentleman"]
    got = group_normalised(mixed)[0]
    assert got == ["Listen", "Silent",
                                        "enlist"]
    assert normalise("Café!") == "cafe"
    # anagrams always go to the same worker
    assert shard_of("eat", 8) == shard_of("tea", 8)
```

OFFICIAL PAGE TEXT

Given an array of strings, group together the words that are anagrams (e.g. gas, sag) of each other. The order of the groups or the words within the groups does not matter in the final result.

MEDIUM | IN APP LC 198 | Exponent | 35 of 36

★ TOP 75: rank 74 in the Medium and Hard priority list

★ Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.2 | 30 DAYS 0 | PRIORITY 51/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #198)

LeetCode match: LC 198 House Robber (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4646/house-robber

QUESTION

Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

APPROACH

dp: rob = skip_prev + x, skip = max(rob_prev, skip_prev); O(n) time, O(1) space.

HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

Basis: LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given n houses in a line with money in
# each, find the maximum amount a robber can steal,
# without stealing from two adjacent houses.
#
# Given n houses in a line with money in each, find the
# maximum amount a robber can steal, without stealing from
# two adjacent houses.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.
#
# BRUTE FORCE (say it first): Recursion over take or skip
# At index i either skip it (go to i + 1) or take it and
# jump to i + 2, and keep the larger total. Exact and the
# oracle; the same suffixes are solved over and over. Time
# O(2^n). Space O(n).


# BRUTE FORCE (say it first): Recursion over take or skip
# | time O(2^n)
def _house_brute(nums):
    def go(i):
        if i >= len(nums):
            return 0
        return max(go(i + 1), nums[i] + go(i + 2))

    return go(0)


# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take = best sum if we take this house; skip = if we
    # do not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last houses touch: ignore one of them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))


# FOLLOW-UP 2: Return which houses to rob.
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i to the end (filled from
    # the right)
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    houses, i = [], 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            houses.append(i)
            i += 2
        else:
            i += 1
    return dp[0], houses


# FOLLOW-UP 3: Robbing a house also alerts the next k
# houses.
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_alarm(nums, k):
    # robbing a house alerts the next k houses
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


if __name__ == "__main__":
    assert rob_line([1, 2, 3, 1]) == 4
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([5]) == 5
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
    assert rob_alarm([2, 7, 9, 3, 1], 1) == 12
    assert rob_alarm([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

OFFICIAL PAGE TEXT

You are given an array representing the value in the houses e.g. [100, 20, 40, 70, 80]. Return 100 + 40 + 80 = 220

Easy (1)

EASY | Exponent | 36 of 36

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 160/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 500 Keyboard Row (inferred from short statement)

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/7293/return-words-one-keyboard-row

QUESTION

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

APPROACH

Map each letter to its keyboard row; keep words whose letters all belong to one row. O(total chars).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask about case (treat as case-insensitive), empty strings, and non-letter characters.

BRUTE FORCE TO SAY OUT LOUD

For each word test all three rows with string containment. Fine and short.

OPTIMAL IDEA

Map each letter to its keyboard row once (26 entries). A word qualifies if the set of rows of its lowercased letters has one element. O(total characters).

DRY-RUN THESE CASES

Mixed case such as Alaska, a word using two rows, an empty word, a one-letter word.

LIKELY FOLLOW-UPS

Different keyboard layouts as a parameter.

PITFALLS

Forgetting to lowercase before looking up rows.

HOW TO THINK ABOUT IT

Each letter belongs to exactly one row, so the question for a word is whether all its letters map to the same row. Build the letter-to-row lookup once and check each word with a set of rows.

Basis: LeetCode 500, Keyboard Row.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
ROWS = [set("qwertyuiop"), set("asdfghjkl"),
        set("zxcvbnm")]
def brute(words):
    found = []
    for word in words:
        letters = set(word.lower())
        for row in ROWS:
            if letters <= row:
                found.append(word)
                break
    return found
```

TIME	SPACE
O(W * L * 3)	O(1)

WHY IT IS SLOW

Up to three scans per word; building one letter-to-row table makes each word a single scan, O(total characters).

OPTIMAL SOLUTION · Map each letter to its row and compare sets

TIME	SPACE
O(total characters)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?

Nothing changes except the row table, which is built from the layout argument. Build the letter-to-row map once and reuse it across queries.

FOLLOW-UP 2 Other layouts (Dvorak, AZERTY) or a phone keypad.

Pass the rows as a parameter and build the letter-to-row dict from it. Nothing else changes.

FOLLOW-UP 3 A different keyboard layout.

Pass the rows as a parameter and build the letter-to-row map from it (a phone keypad or Dvorak works the same way). Letters that are on no row make the word invalid, which the code checks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given an array of strings, return the
# words that can be typed using letters of the alphabet on
# only one row of American keyboard.
#
# Given an array of strings, return the words that can be
# typed using letters of the alphabet on only one row of
# American keyboard.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the keyboard layout is a parameter (Dvorak,
# AZERTY, a phone keypad)?
# 2. Other layouts (Dvorak, AZERTY) or a phone keypad.
# 3. A different keyboard layout.
#
# INTERVIEW NOTES
# Each letter belongs to exactly one row, so the question
# for a word is whether all its letters map to the same
# row. Build the letter-to-row lookup once and check each
# word with a set of rows.
#
# BRUTE FORCE (say it first): For each word test whether
# it fits each row
# For every word, check whether all of its lowercase
# letters are in the first row, or all in the second, or
# all in the third. Correct; it rescans each word up to
# three times. Time O(W * L * 3). Space O(1).


# BRUTE FORCE (say it first): For each word test whether
# it fits each row | time O(W * L * 3)
def keyboard_row_brute(words):
    rows = [
        set("qwertyuiop"),
        set("asdfghjkl"),
        set("zxcvbnm"),
    ]
    return [
        w
        for w in words
        if any(set(w.lower()) <= row for row in rows)
    ]


# HELPERS for the base question
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]
```

```
# LAYOUT table (change it for other keyboards)
def row_map(rows):
    # letter -> index of its row; built once and reused
    return {ch: i for i, row in enumerate(rows)
            for ch in row}


# BASE QUESTION (optimal): Map each letter to its row and
# compare sets
# Idea: Build a dict from letter to row index (26
# entries). A word qualifies if all of its lowercased
# letters map to the same row, i.e. the set of rows has
# size one. Lowercase first so 'Alaska' works.
# FOLLOW-UP 3: A different keyboard layout.
# Answer: Pass the rows as a parameter and build the
# letter-to-row map from it (a phone keypad or Dvorak
# works the same way). Letters that are on no row make the
# word invalid, which the code checks.
```

```
def one_row_words(words, rows=QWERTY):
    # FOLLOW-UP 3: pass rows= for another keyboard layout
    row_of = row_map(rows)
    out = []
    for w in words:
        letters = w.lower()
        # a letter that is on no row makes the word
        # invalid
        if any(c not in row_of for c in letters):
            continue
        # all letters must come from the same row
        if len({row_of[c] for c in letters}) <= 1:
            out.append(w)
    return out


if __name__ == "__main__":
    words = ["Hello", "Alaska", "Dad", "Peace"]
    assert one_row_words(words) == ["Alaska", "Dad"]
    assert one_row_words([]) == []
    assert one_row_words(["", "a1"]) == [""]
    # another layout: a phone keypad
    keypad = ["abc", "def", "ghi"]
    got = one_row_words(["bad", "fed", "cab"], keypad)
    assert got == ["fed", "cab"]
```

```
    # Dvorak top row
    dvorak = ["pyfgcrl", "aoeuidhtns", "qjkxbmwvz"]
    assert one_row_words(["tent"], dvorak) == ["tent"]
```